-- SQL Mini Project - Retail Store

DDL SCRIPTS

```
create database retail_db;

use retail_db;

CREATE TABLE customers (

    customer_id INT PRIMARY KEY AUTO_INCREMENT,

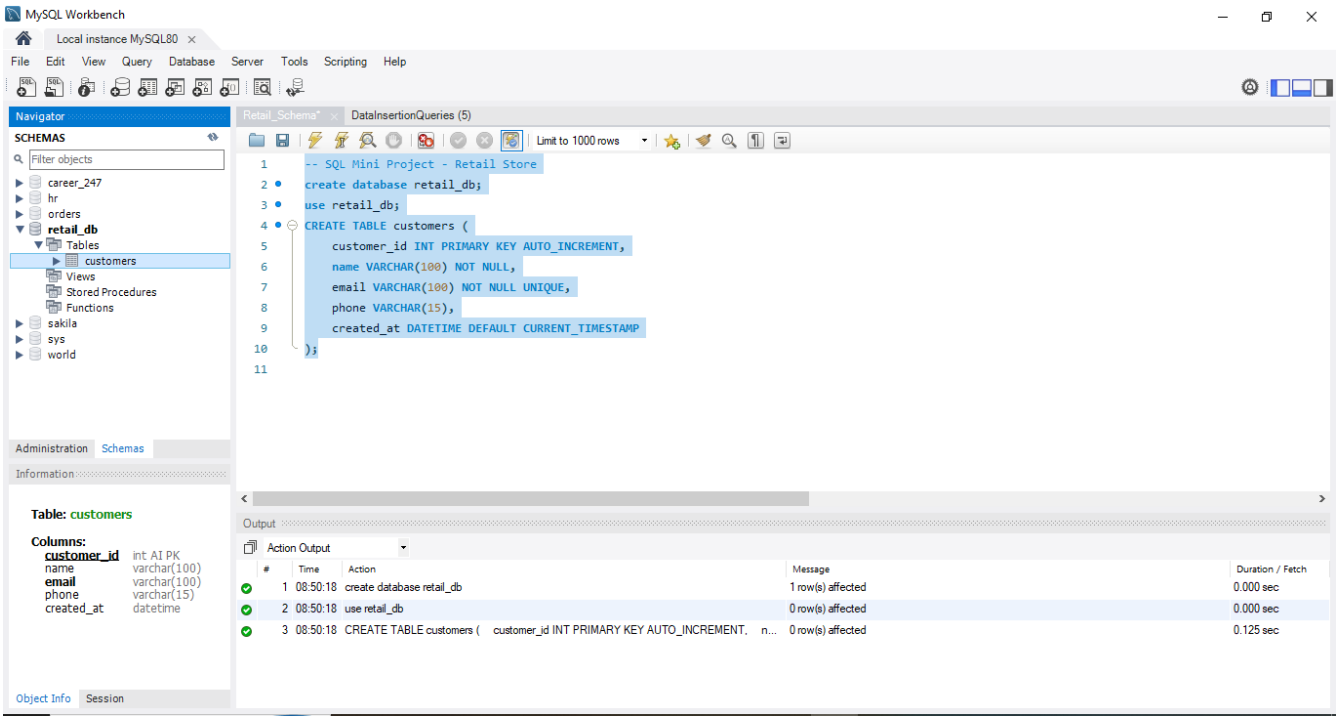
    name VARCHAR(100) NOT NULL,

    email VARCHAR(100) NOT NULL UNIQUE,

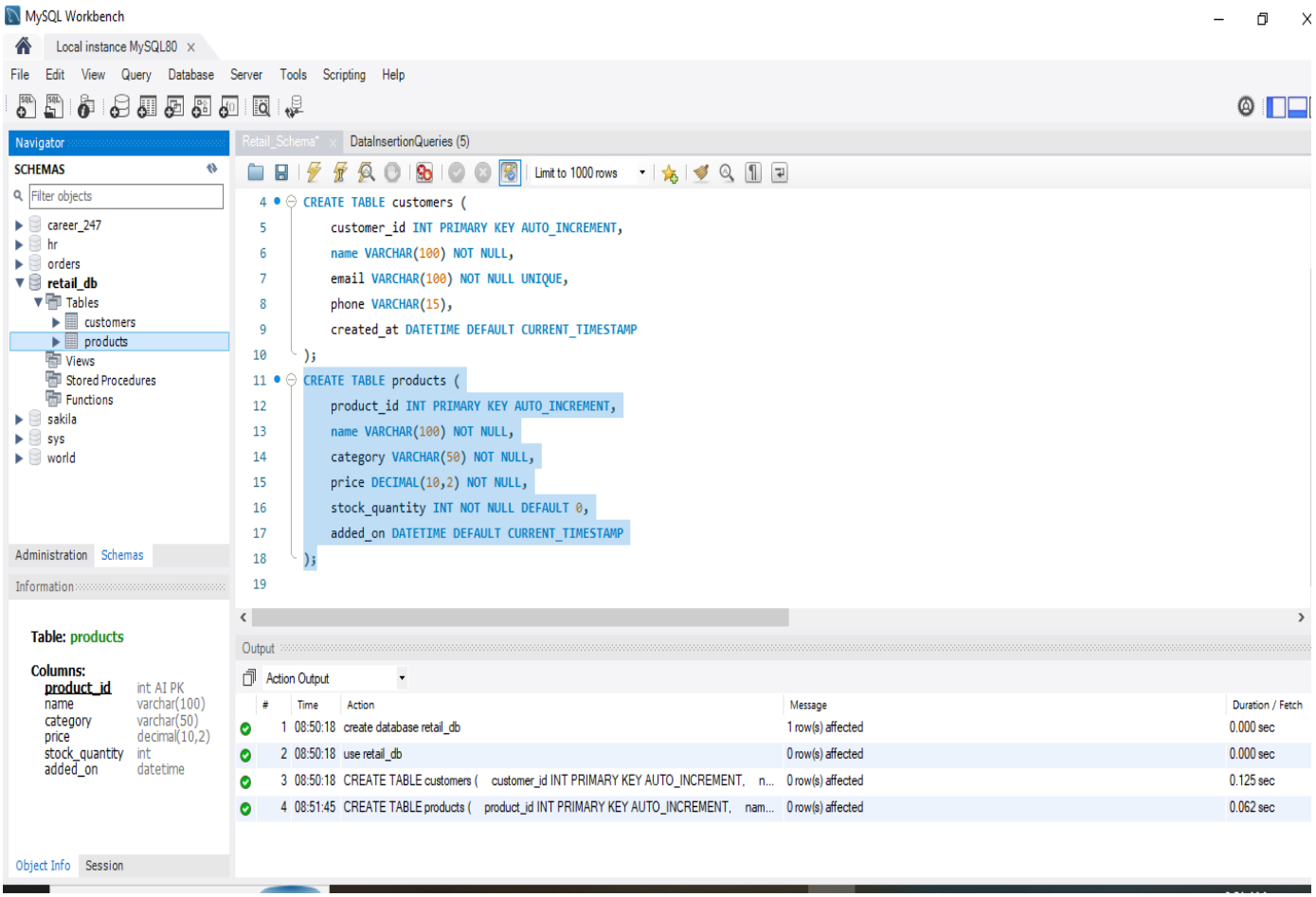
    phone VARCHAR(15),

    created_at DATETIME DEFAULT CURRENT_TIMESTAMP

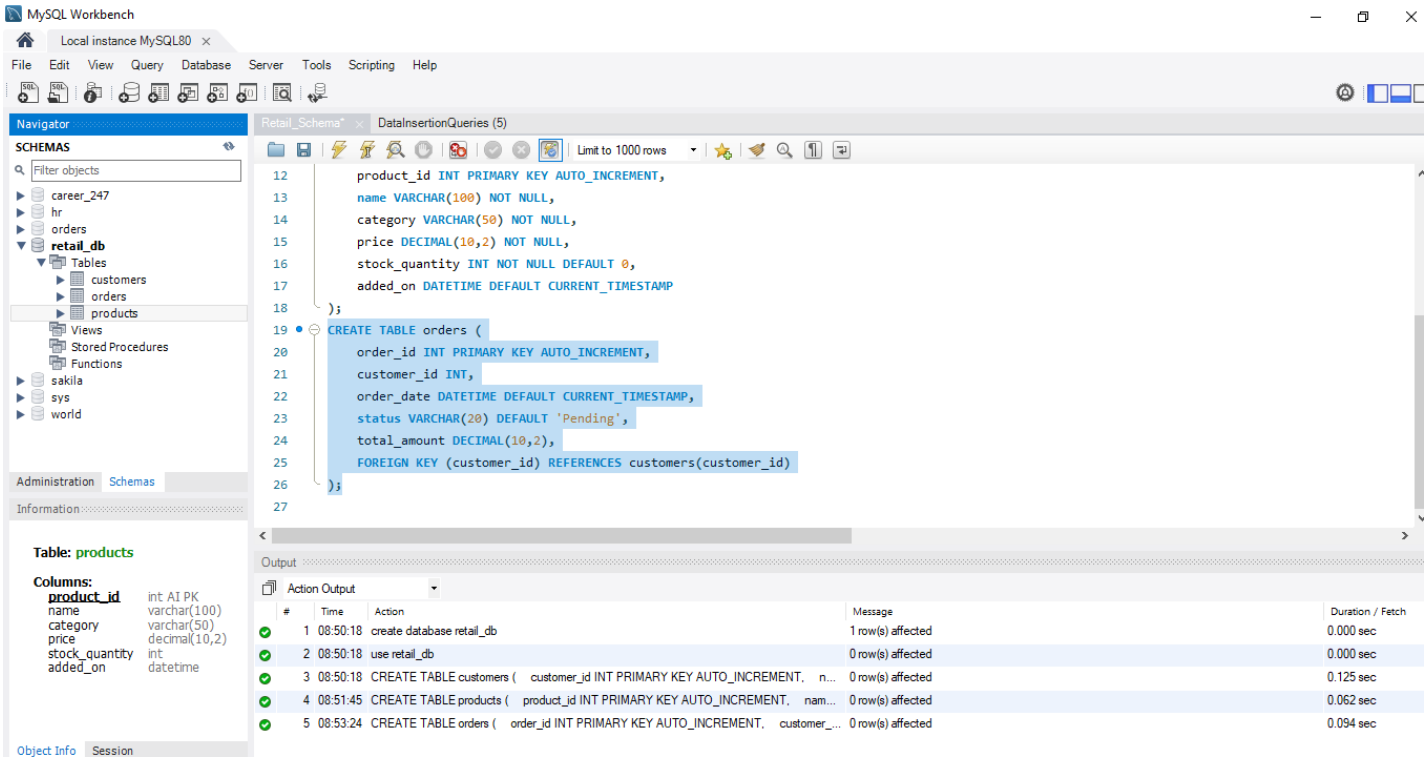
);
```



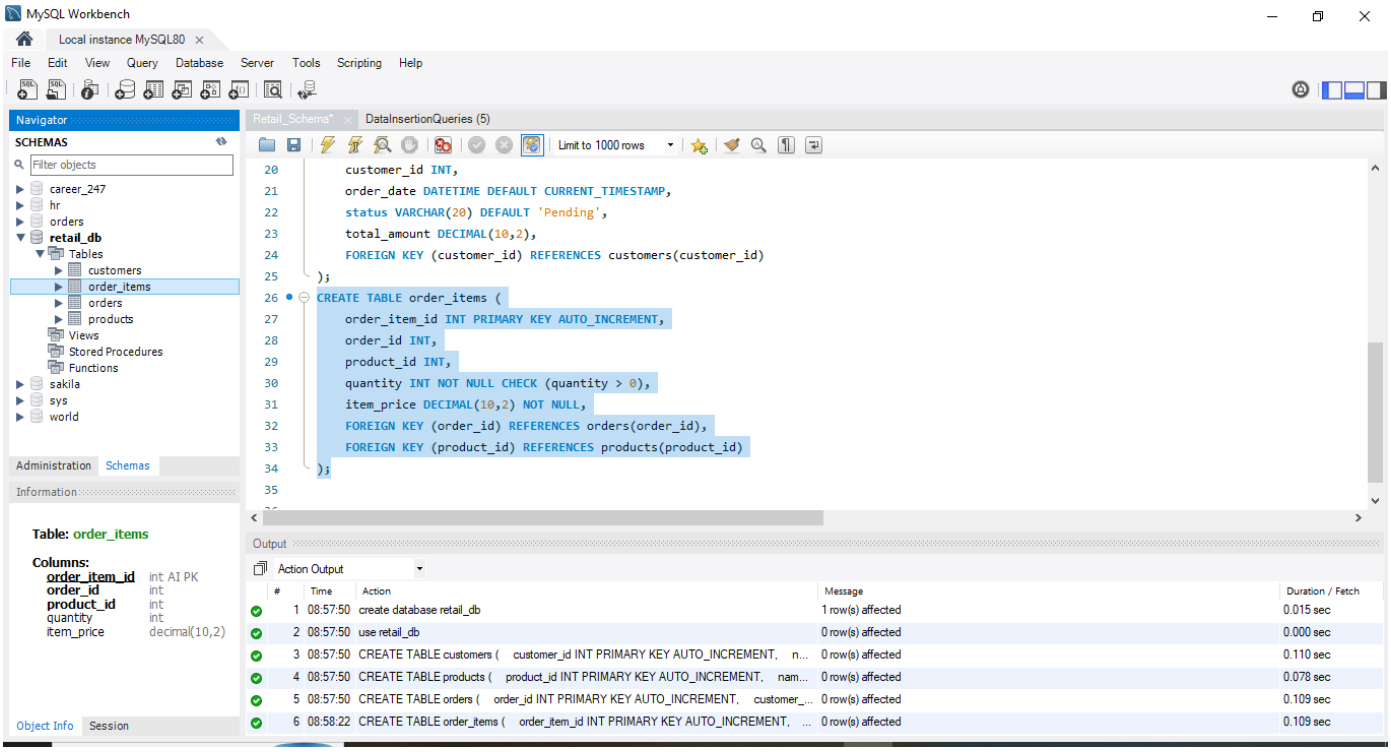
```
CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT NOT NULL DEFAULT 0,
    added_on DATETIME DEFAULT CURRENT_TIMESTAMP
);
```



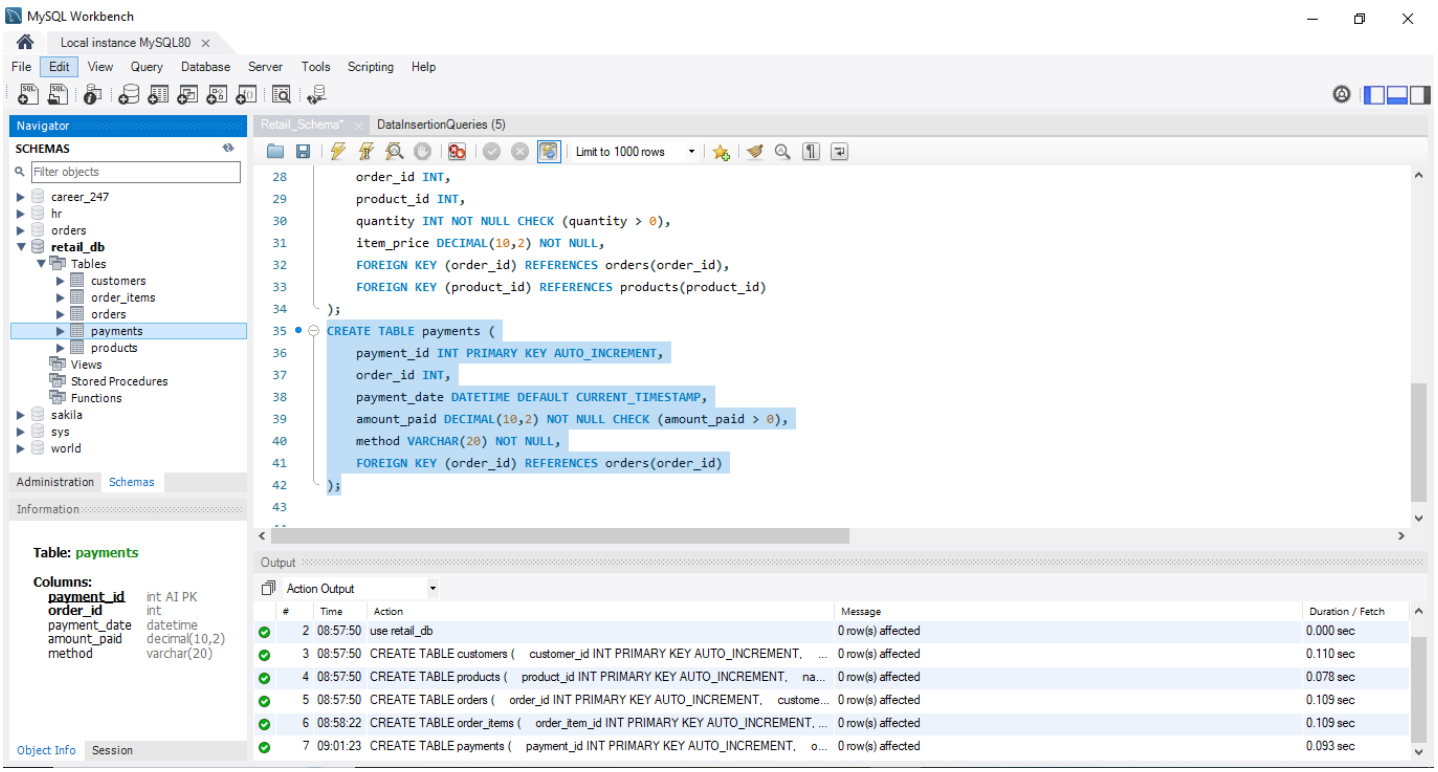
```
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20) DEFAULT 'Pending',
    total_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```



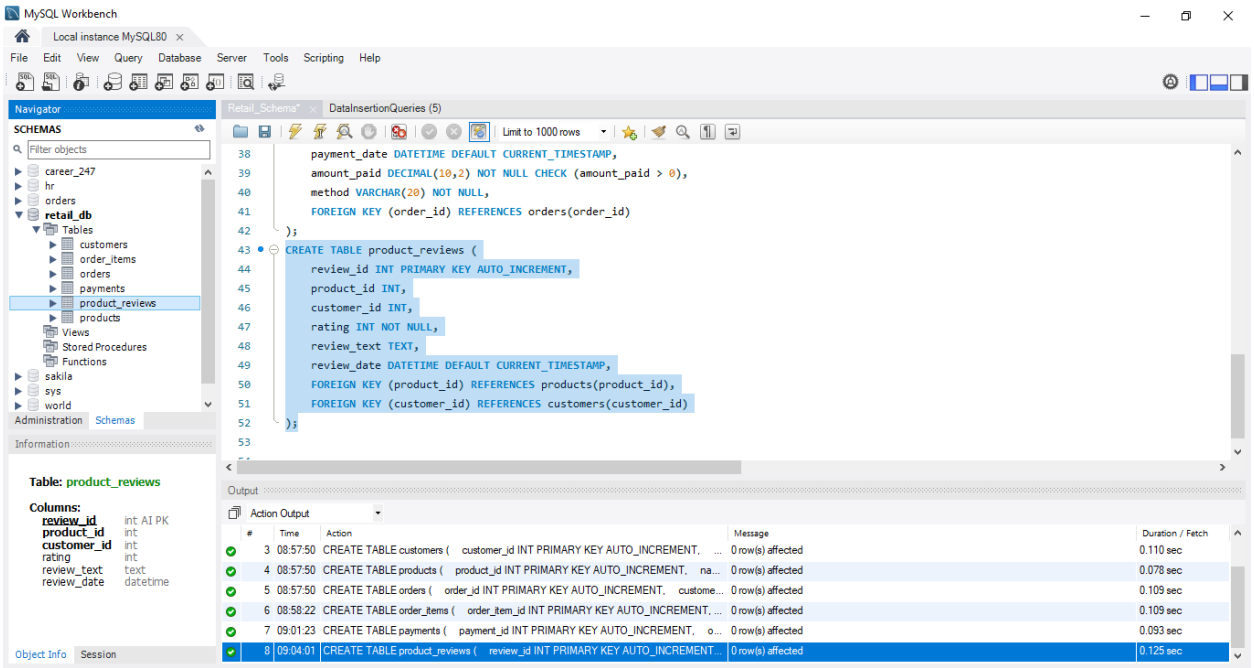
```
CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT,
    product_id INT,
    quantity INT NOT NULL CHECK (quantity > 0),
    item_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

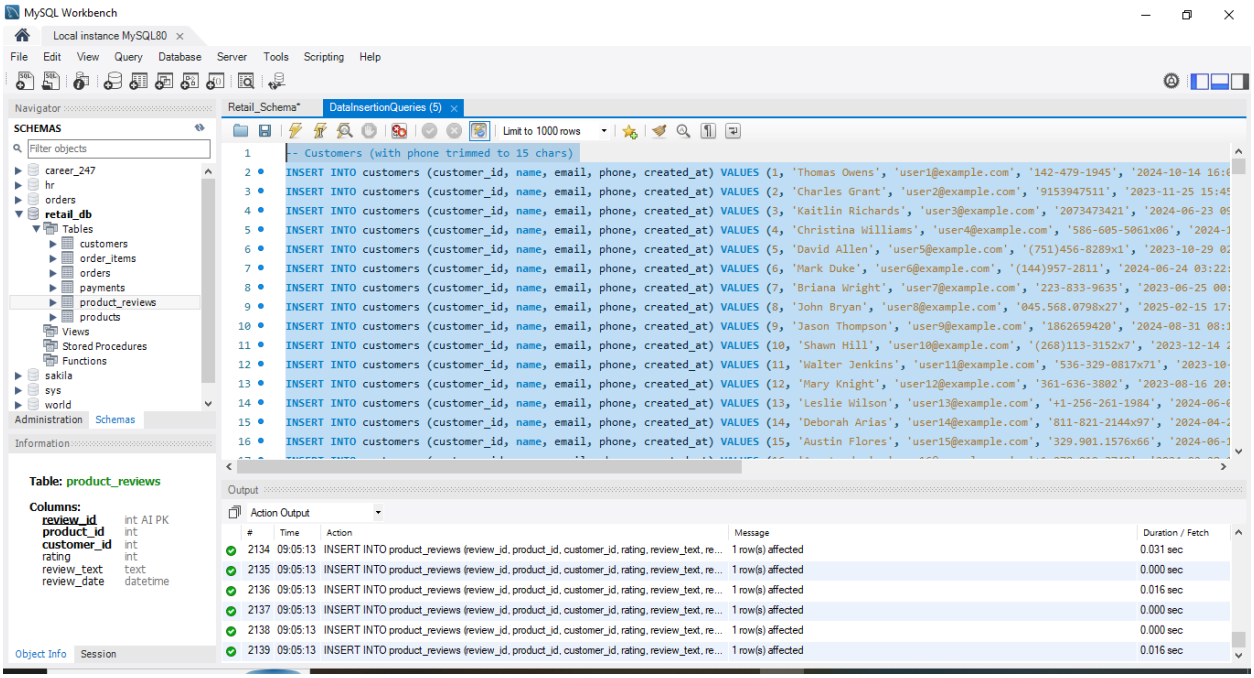


```
CREATE TABLE payments (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT,
    payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    amount_paid DECIMAL(10,2) NOT NULL CHECK (amount_paid > 0),
    method VARCHAR(20) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
```

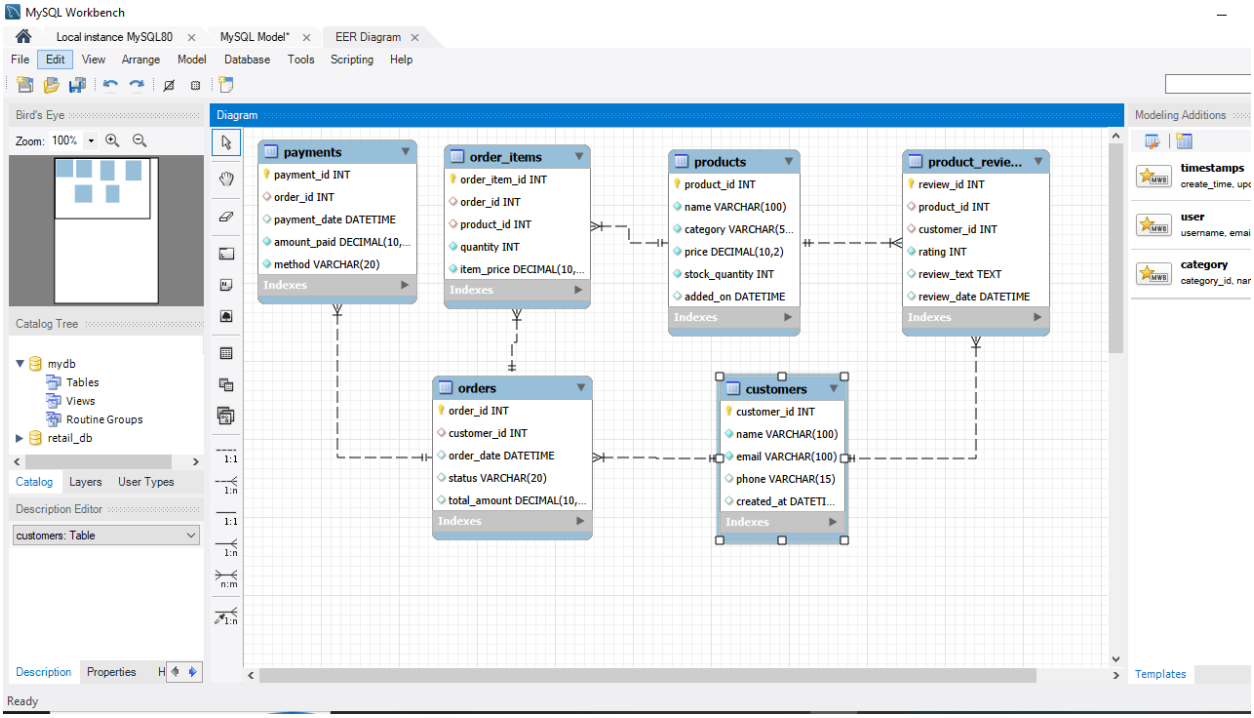


```
CREATE TABLE product_reviews (
    review_id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT,
    customer_id INT,
    rating INT NOT NULL,
    review_text TEXT,
    review_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES products(product_id),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```




MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
career_247
hr
orders
retail_db
Tables
customers
order_items
orders
payments
product_reviews
products
Views
Stored Procedures
Functions
sakila
sys
world
Administration Schemas
Information
Table: product_reviews
Columns:
review_id int AI PK
product_id int
customer_id int
rating int
review_text text
review_date datetime
Object Info Session
Retail_Schema*
DataInsertionQueries (5)
Limit to 1000 rows
-- Customers (with phone trimmed to 15 chars)
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (1, 'Thomas Owens', 'user1@example.com', '142-479-1945', '2024-10-14 16:
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (2, 'Charles Grant', 'user2@example.com', '9153947511', '2023-11-25 15:4
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (3, 'Kaitlin Richards', 'user3@example.com', '2073473421', '2024-06-23 0
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (4, 'Christina Williams', 'user4@example.com', '586-605-5061x06', '2024-1
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (5, 'David Allen', 'user5@example.com', '(751)456-8289x1', '2023-10-29 0
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (6, 'Mark Duke', 'user6@example.com', '(144)957-2811', '2024-06-24 03:22:
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (7, 'Briana Wright', 'user7@example.com', '223-833-9635', '2023-06-25 00:
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (8, 'John Bryan', 'user8@example.com', '045.568.0798x27', '2025-02-15 17:
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (9, 'Jason Thompson', 'user9@example.com', '1862659420', '2024-08-31 08:1
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (10, 'Shawn Hill', 'user10@example.com', '(268)113-3152x7', '2023-12-14
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (11, 'Walter Jenkins', 'user11@example.com', '536-329-0817x71', '2023-10-
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (12, 'Mary Knight', 'user12@example.com', '361-636-3802', '2023-08-16 20:
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (13, 'Leslie Wilson', 'user13@example.com', '+1-256-261-1984', '2024-06-
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (14, 'Deborah Arias', 'user14@example.com', '811-821-2144x97', '2024-04-
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (15, 'Austin Flores', 'user15@example.com', '329.901.1576x66', '2024-06-
Output
Action Output
Time Action Message Duration / Fetch
2134 09:05:13 INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, re... 1 row(s) affected 0.031 sec
2135 09:05:13 INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, re... 1 row(s) affected 0.000 sec
2136 09:05:13 INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, re... 1 row(s) affected 0.016 sec
2137 09:05:13 INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, re... 1 row(s) affected 0.000 sec
2138 09:05:13 INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, re... 1 row(s) affected 0.000 sec
2139 09:05:13 INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, re... 1 row(s) affected 0.016 sec

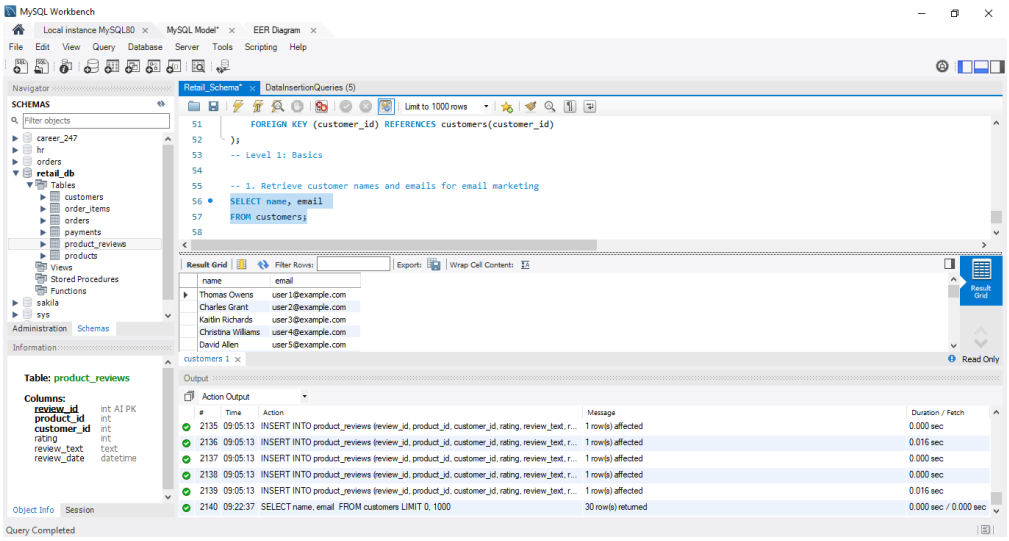
EER DIAGRAM


MySQL Workbench
Local instance MySQL80
MySQL Model*
EER Diagram
File Edit View Arrange Model Database Tools Scripting Help
Bird's Eye
Zoom: 100%
Diagram
payments
payment_id INT
order_id INT
payment_date DATETIME
amount_paid DECIMAL(10,...
method VARCHAR(20)
Indexes
order_items
order_item_id INT
order_id INT
product_id INT
quantity INT
item_price DECIMAL(10,...
Indexes
products
product_id INT
name VARCHAR(100)
category VARCHAR(5...
price DECIMAL(10,2)
stock_quantity INT
added_on DATETIME
Indexes
product_revie...
review_id INT
product_id INT
customer_id INT
rating INT
review_text TEXT
review_date DATETIME
Indexes
orders
order_id INT
customer_id INT
order_date DATETIME
status VARCHAR(20)
total_amount DECIMAL(10,...
Indexes
customers
customer_id INT
name VARCHAR(100)
email VARCHAR(100)
phone VARCHAR(15)
created_at DATETI...
Indexes
Modeling Additions
timestamps
create_time, upd
user
username, emai
category
category_id, nar
Catalog Tree
mydb
Tables
Views
Routine Groups
retail_db
Catalog Layers User Types
Description Editor
customers: Table
Description Properties
Templates
Ready

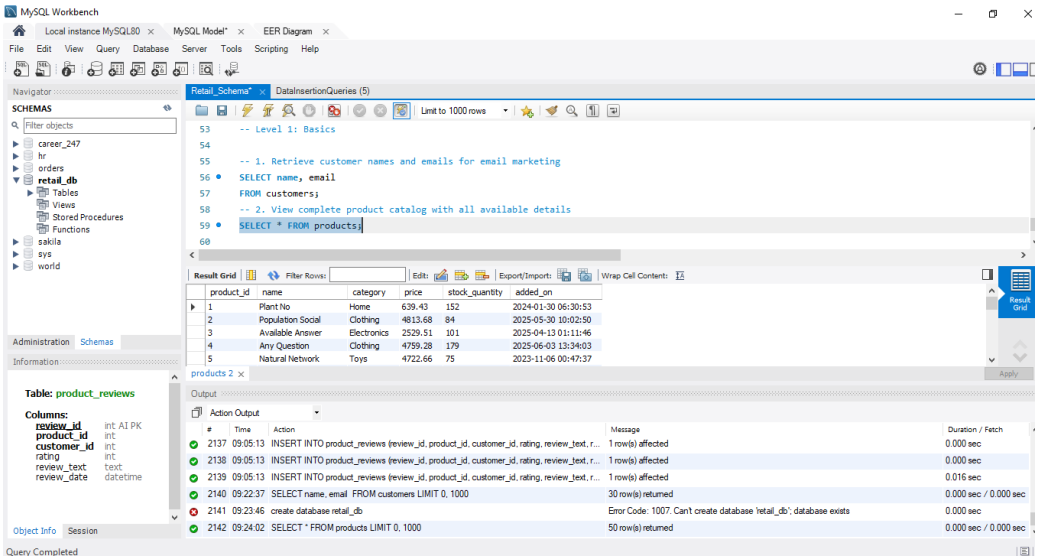
Questions

-- Level 1: Basics

-- 1. Retrieve customer names and emails for email marketing
SELECT name, email
FROM customers;

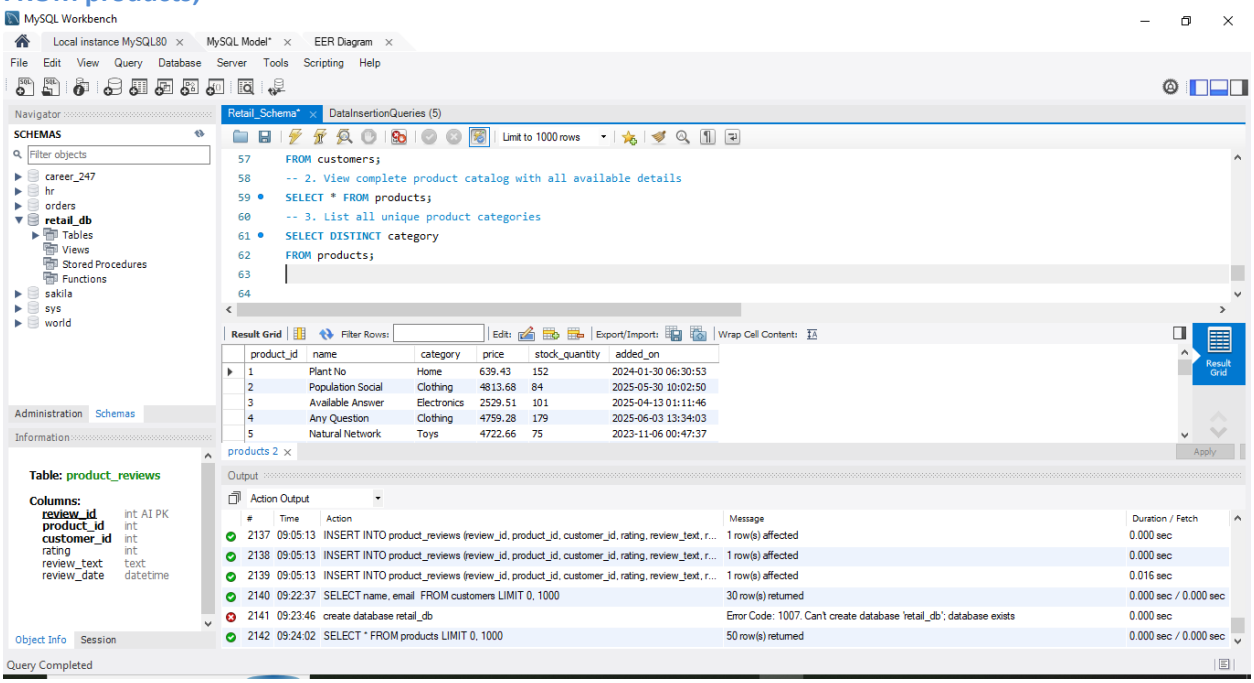


-- 2. View complete product catalog with all available details
SELECT * FROM products;



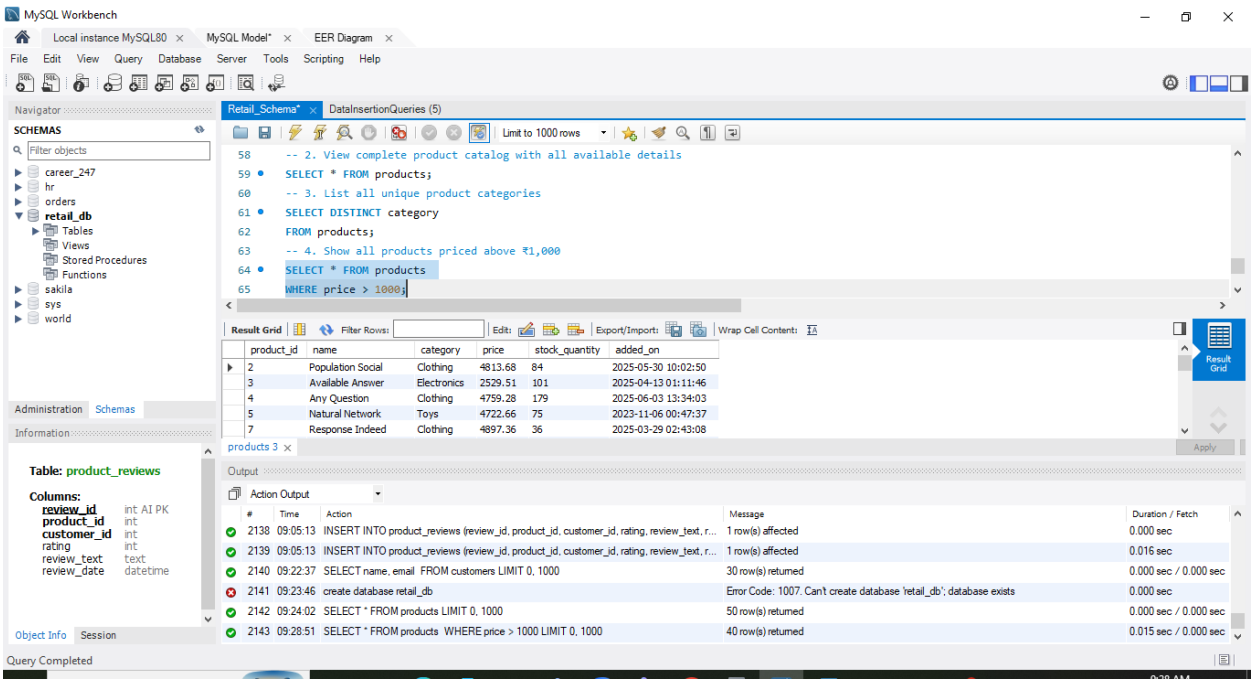
-- 3. List all unique product categories

```
SELECT DISTINCT category
FROM products;
```



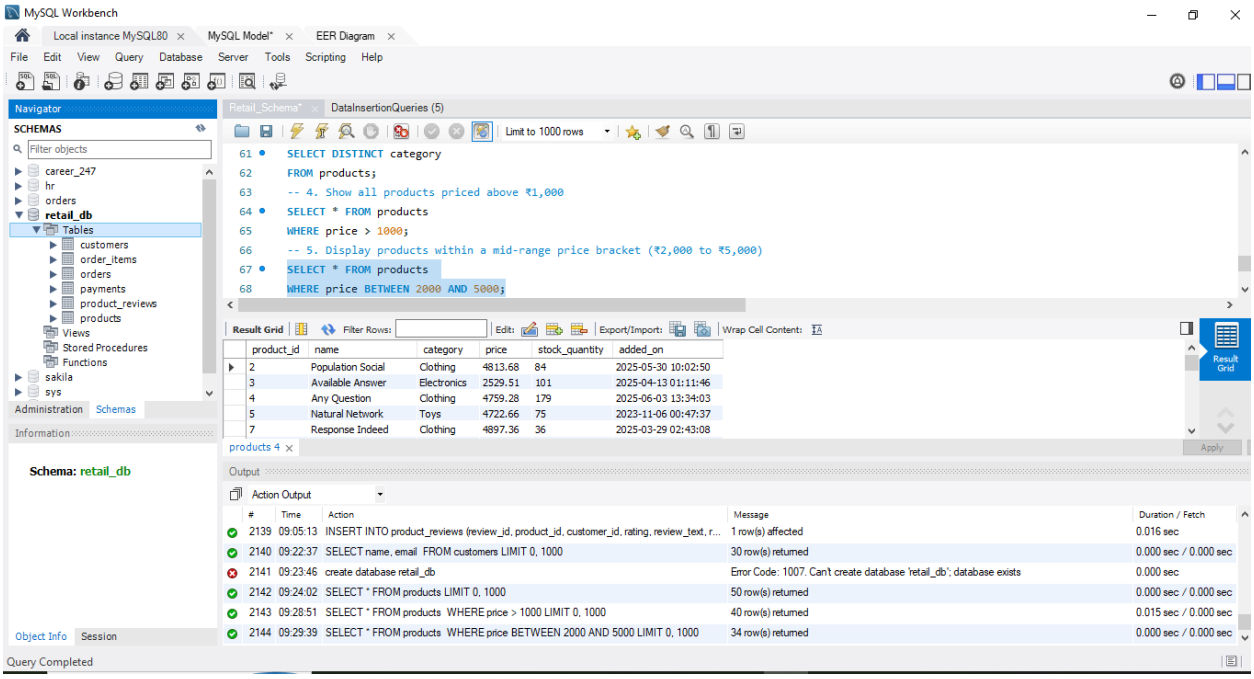
-- 4. Show all products priced above ₹1,000

```
SELECT * FROM products
WHERE price > 1000;
```



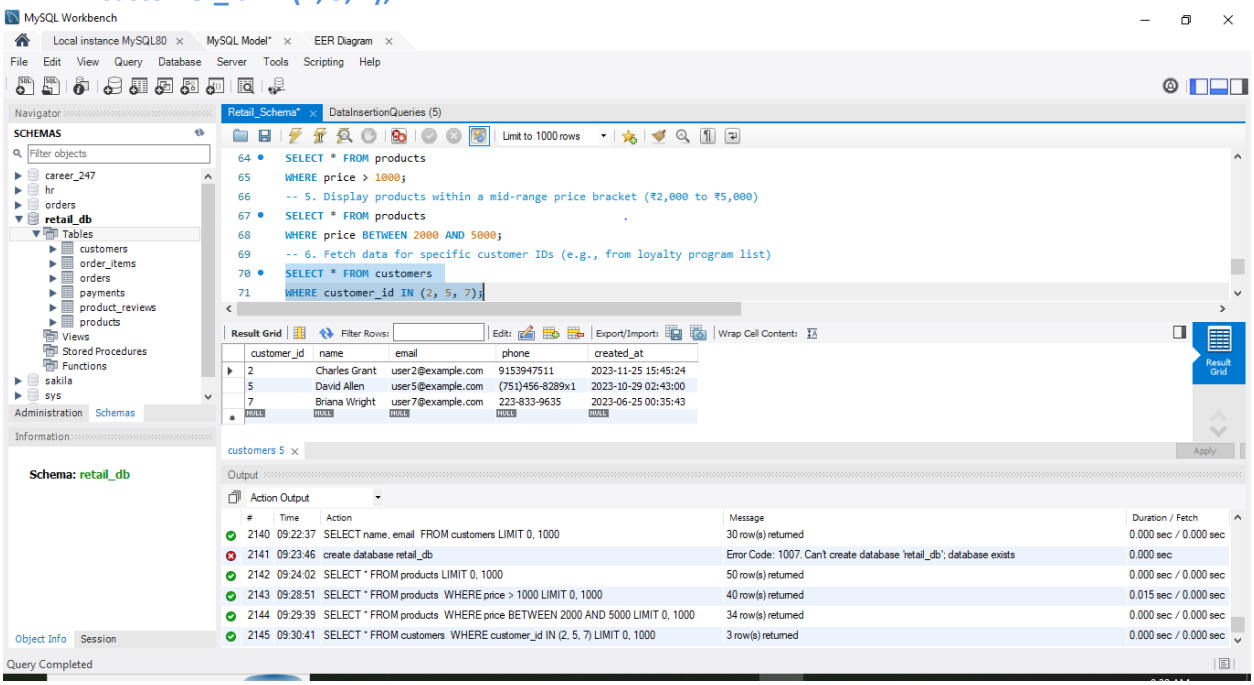
-- 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)

```
SELECT * FROM products
WHERE price BETWEEN 2000 AND 5000;
```



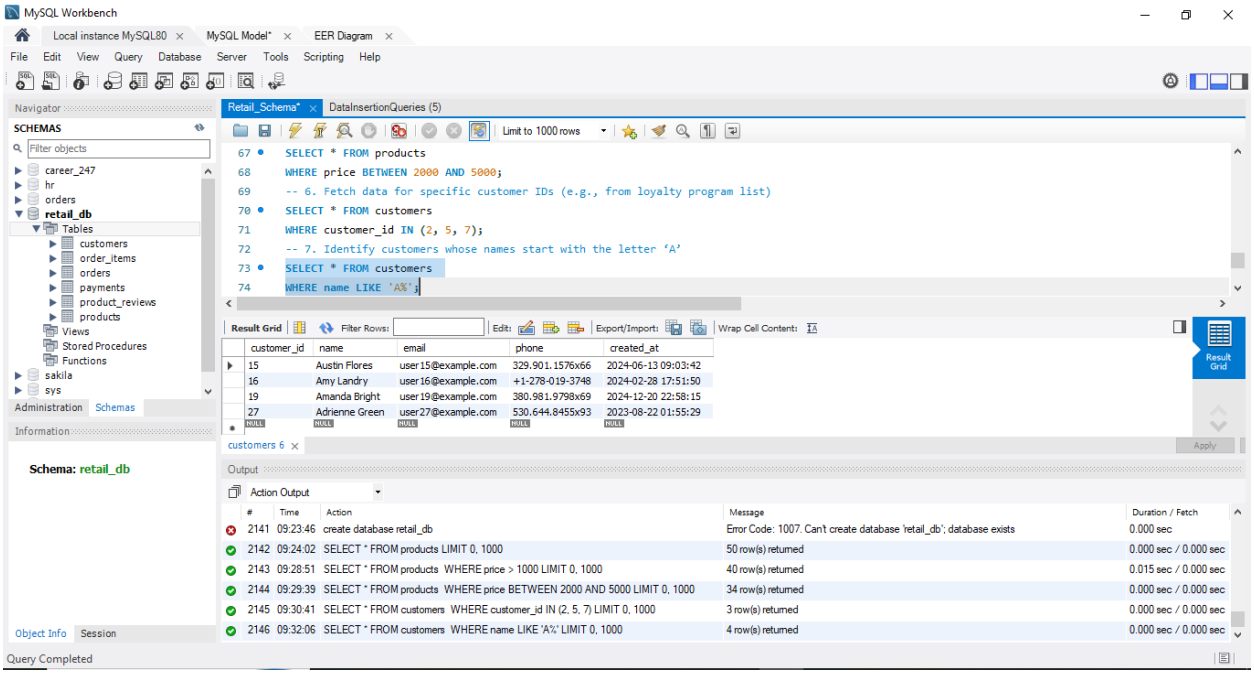
-- 6. Fetch data for specific customer IDs (e.g., from loyalty program list)

```
SELECT * FROM customers
WHERE customer_id IN (2, 5, 7);
```



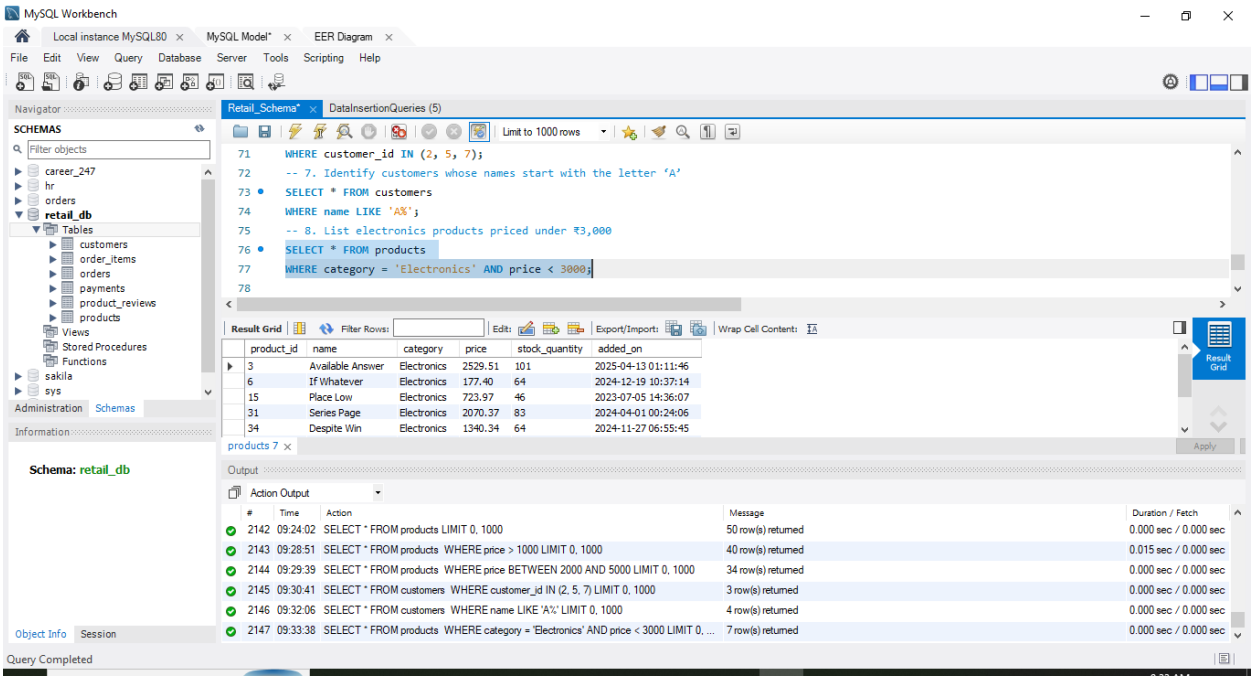
-- 7. Identify customers whose names start with the letter ‘A’

```
SELECT * FROM customers
WHERE name LIKE 'A%';
```



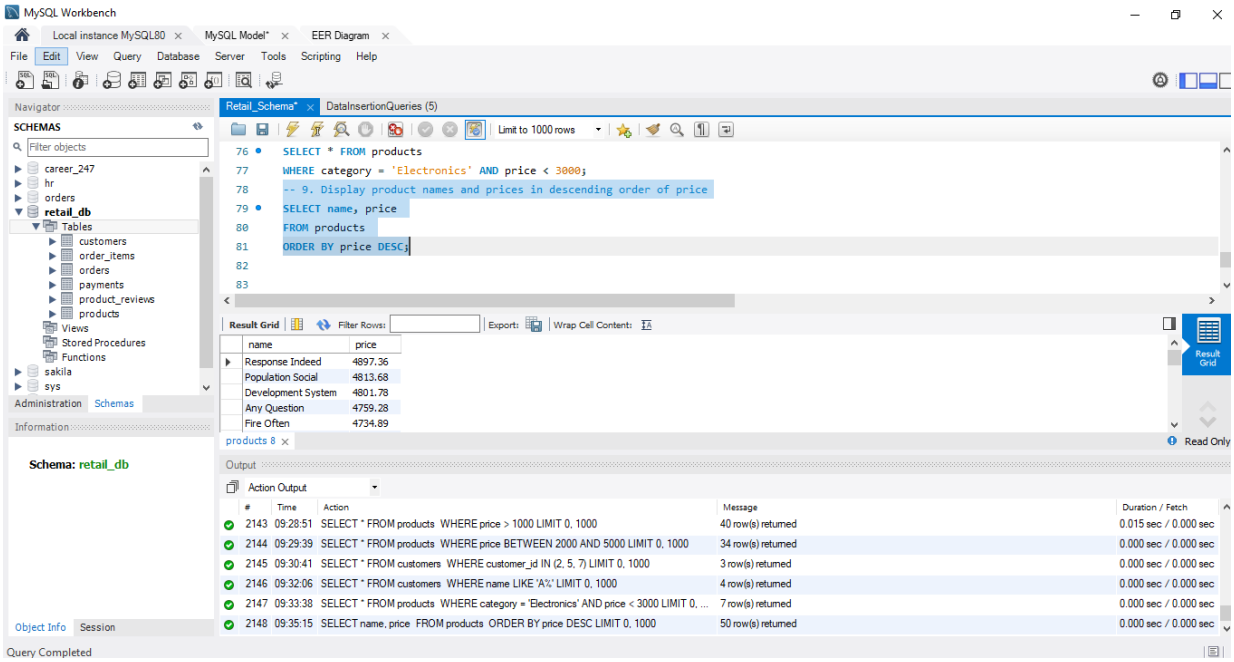
-- 8. List electronics products priced under ₹3,000

```
SELECT * FROM products
WHERE category = 'Electronics' AND price < 3000;
```



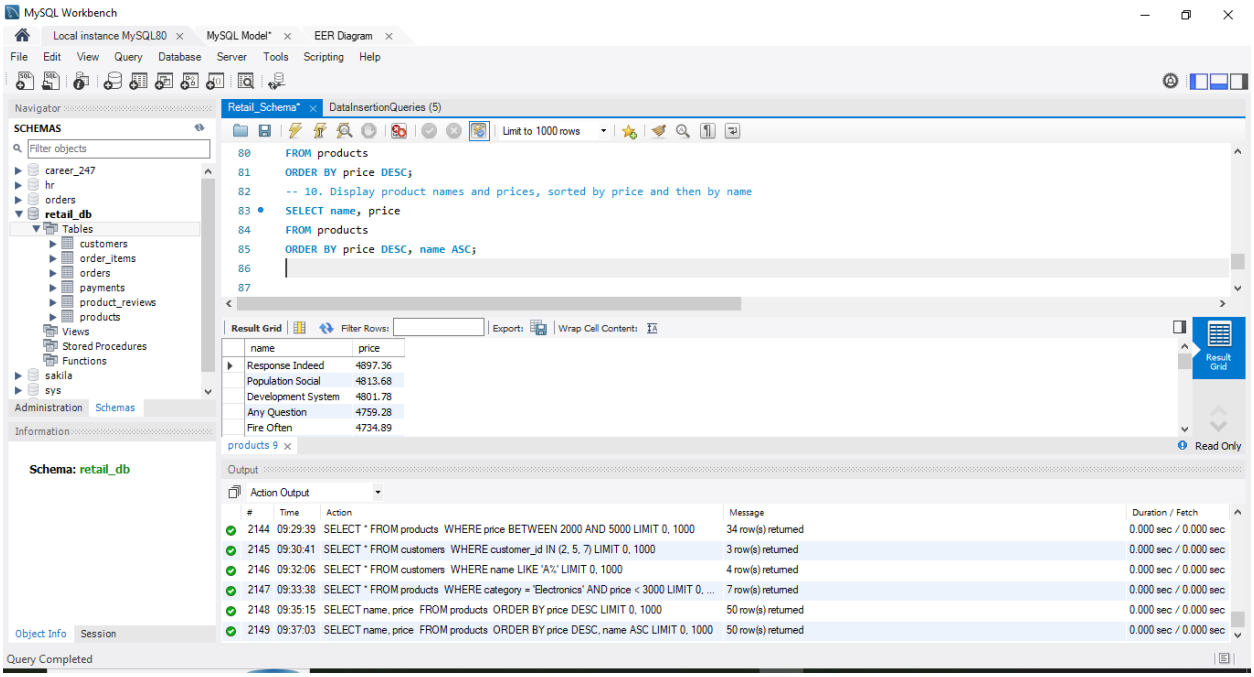
-- 9. Display product names and prices in descending order of price

```
SELECT name, price
FROM products
ORDER BY price DESC;
```



-- 10. Display product names and prices, sorted by price and then by name

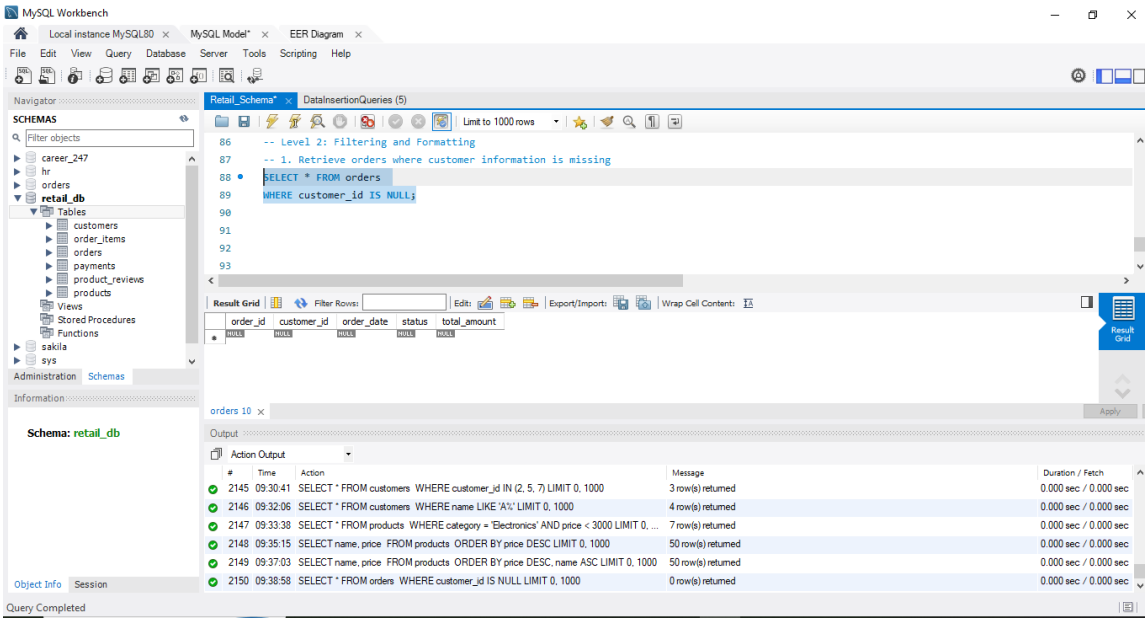
```
SELECT name, price
FROM products
ORDER BY price DESC, name ASC;
```



-- Level 2: Filtering and Formatting

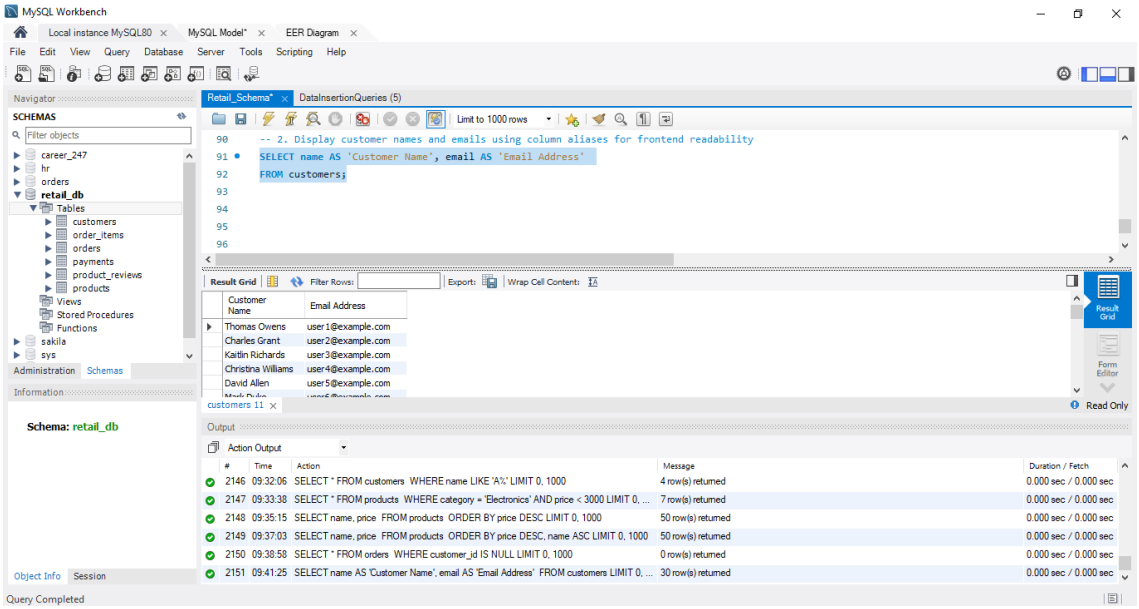
-- 1. Retrieve orders where customer information is missing

```
SELECT * FROM orders
WHERE customer_id IS NULL;
```



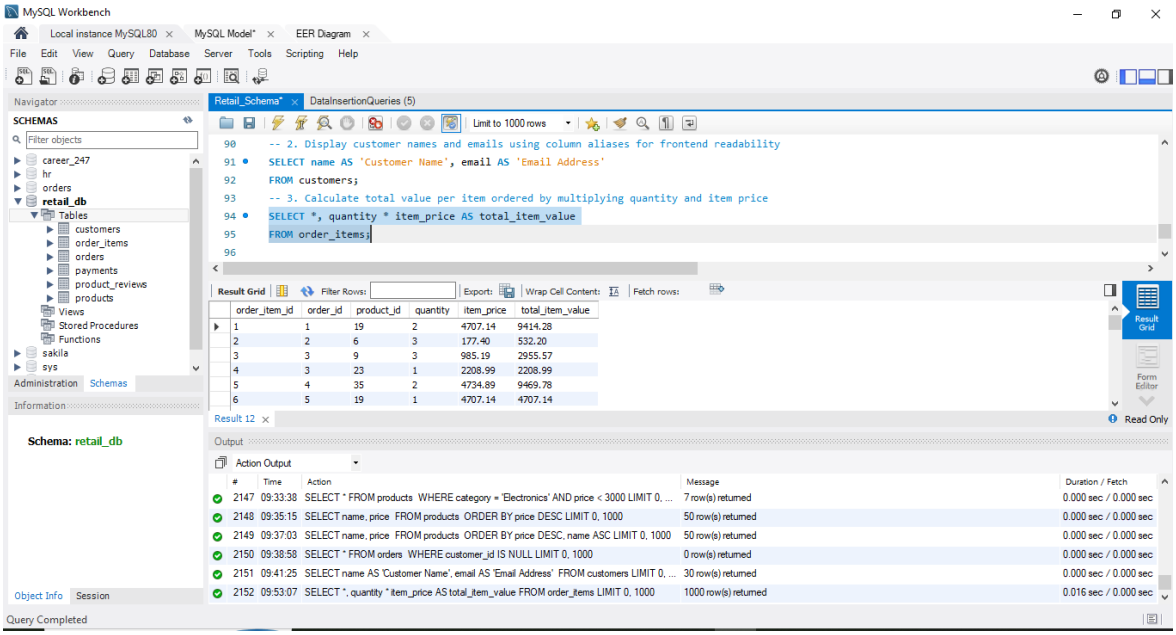
-- 2. Display customer names and emails using column aliases for frontend readability

```
SELECT name AS 'Customer Name', email AS 'Email Address'
FROM customers;
```



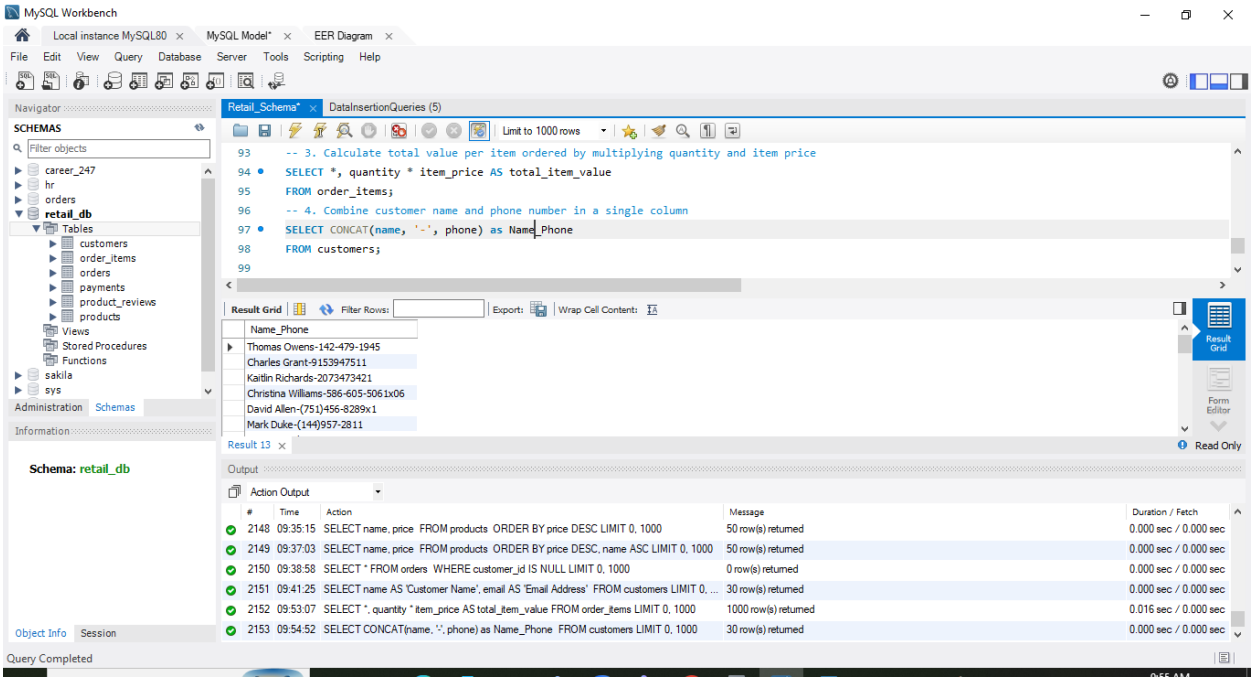
-- 3. Calculate total value per item ordered by multiplying quantity and item price

```
SELECT *, quantity * item_price AS total_item_value
FROM order_items;
```



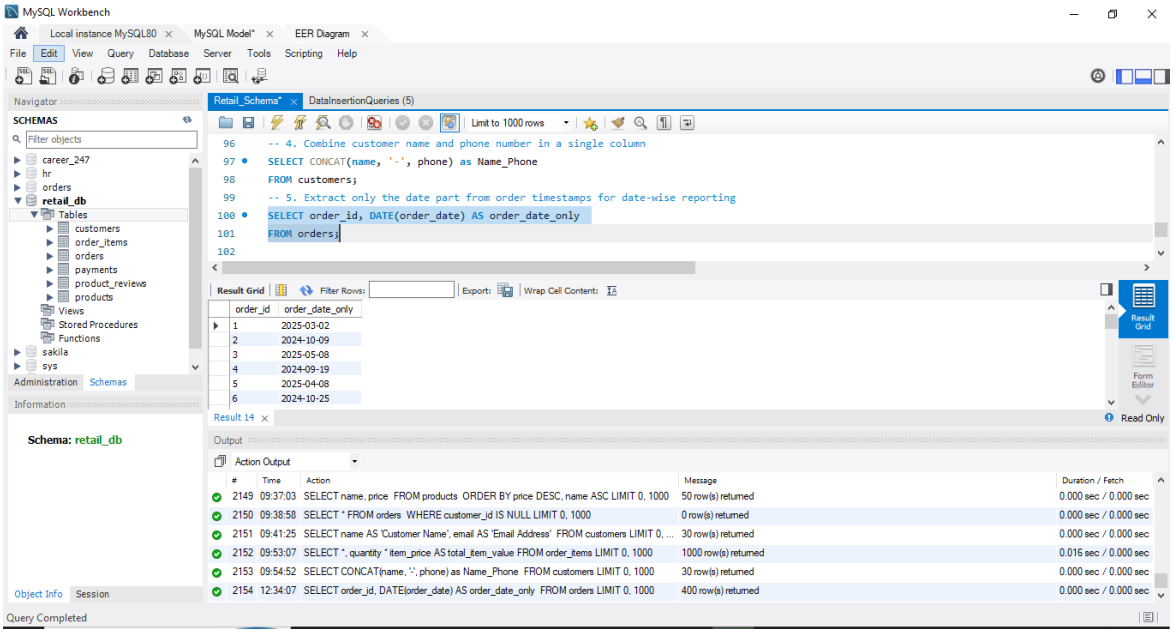
-- 4. Combine customer name and phone number in a single column

```
SELECT CONCAT(name, '-', phone) as Name_Phone
FROM customers;
```



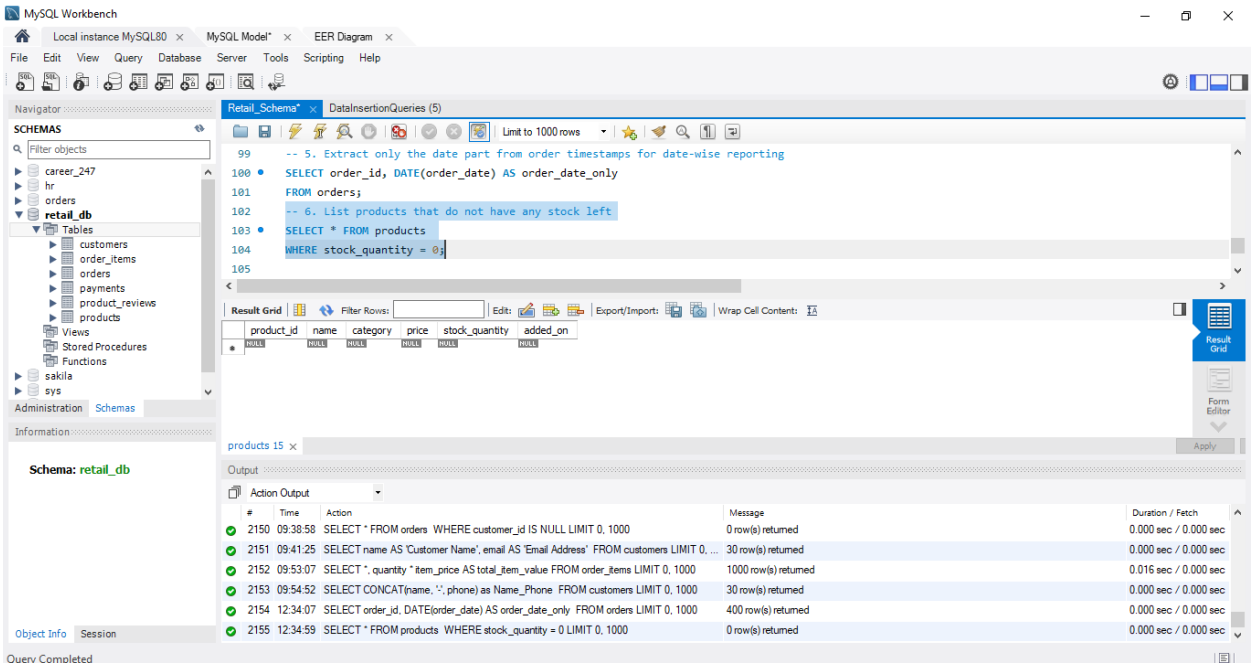
-- 5. Extract only the date part from order timestamps for date-wise reporting

```
SELECT order_id, DATE(order_date) AS order_date_only
FROM orders;
```



-- 6. List products that do not have any stock left

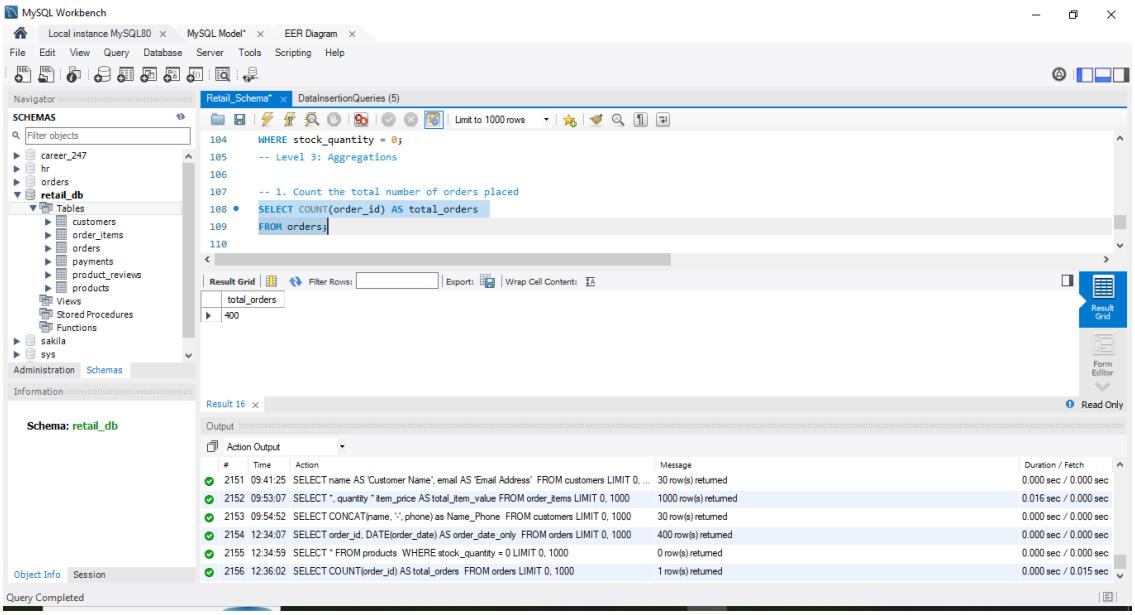
```
SELECT * FROM products
WHERE stock_quantity = 0;
```



-- Level 3: Aggregations

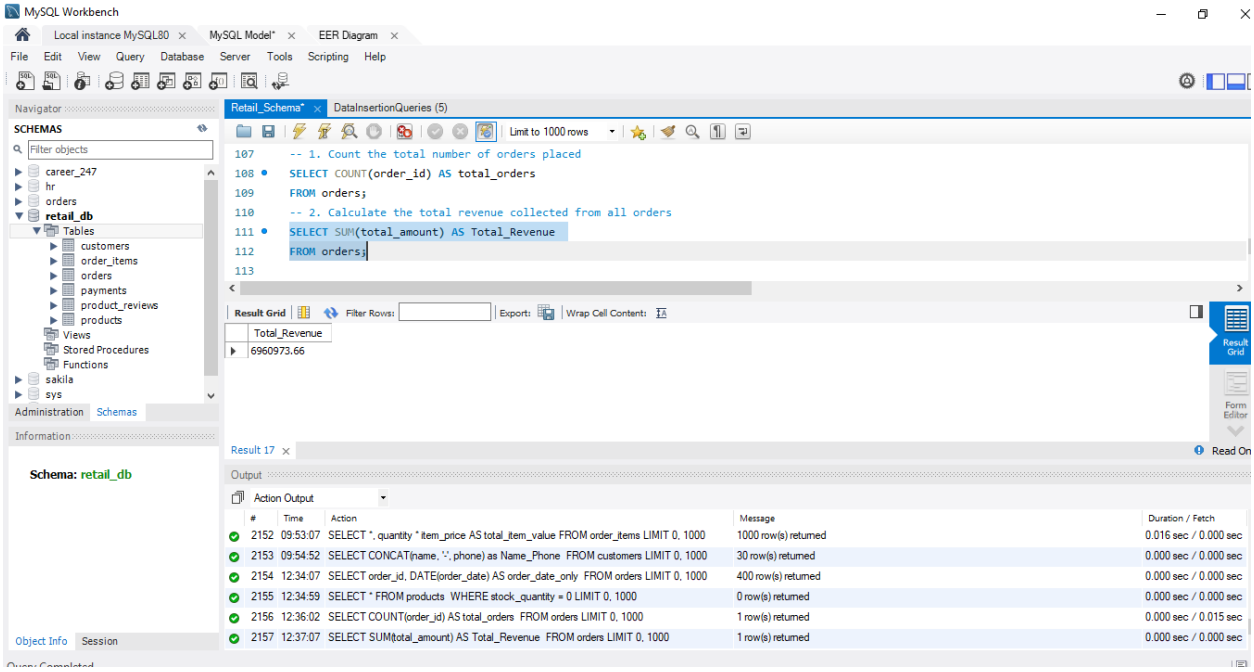
-- 1. Count the total number of orders placed

```
SELECT COUNT(order_id) AS total_orders
FROM orders;
```



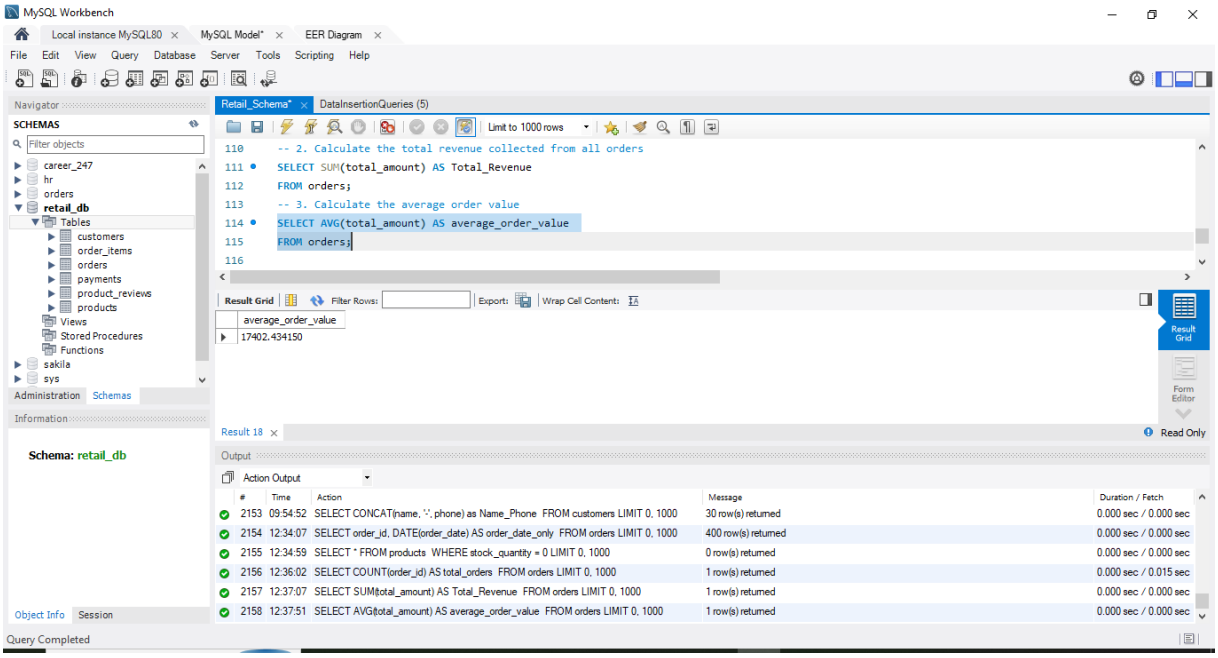
-- 2. Calculate the total revenue collected from all orders

```
SELECT SUM(total_amount) AS Total_Revenue
FROM orders;
```



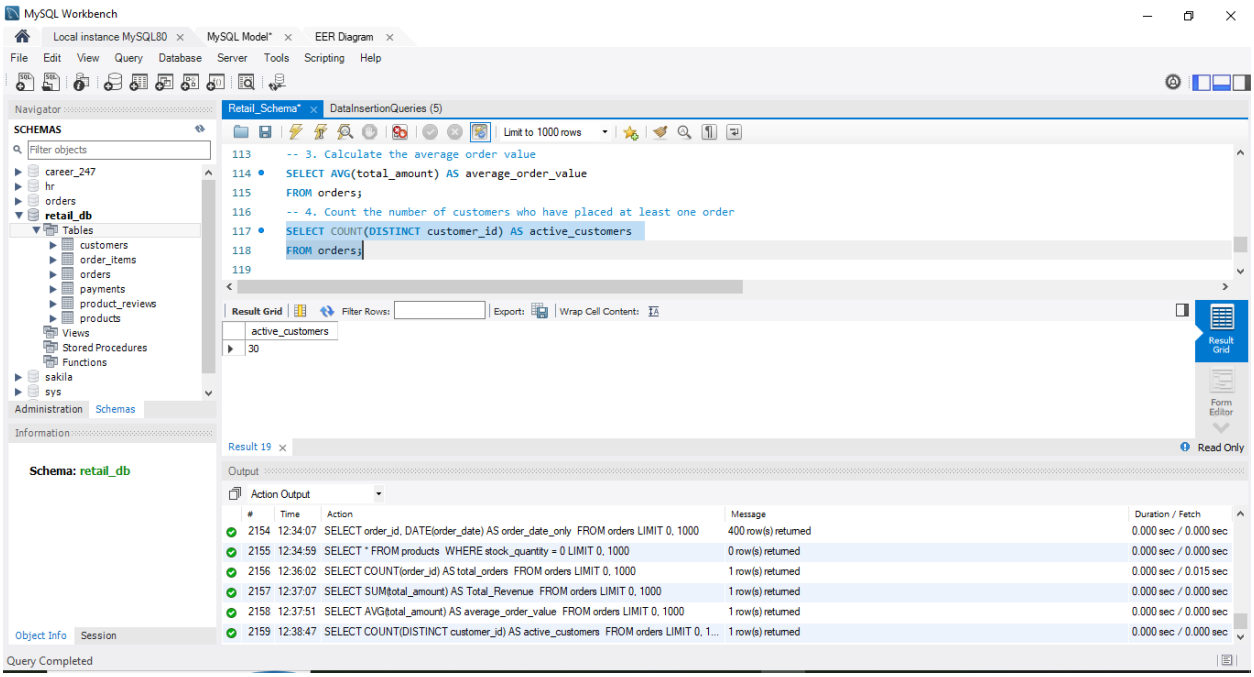
-- 3. Calculate the average order value

```
SELECT AVG(total_amount) AS average_order_value
FROM orders;
```



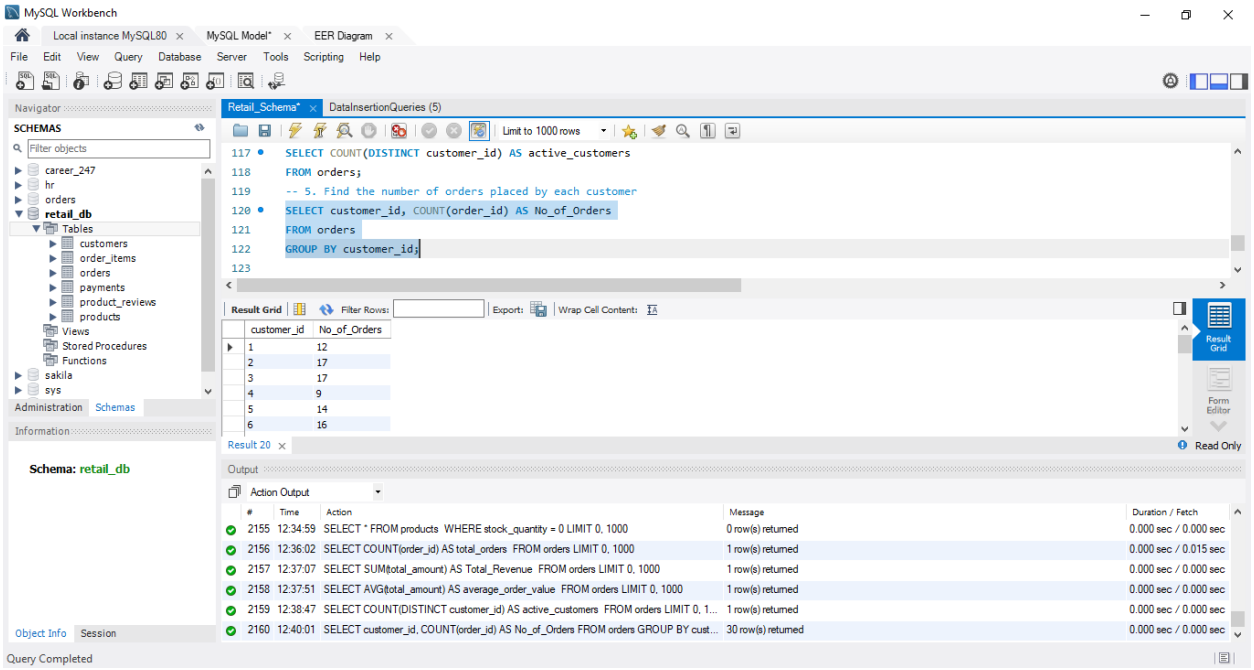
-- 4. Count the number of customers who have placed at least one order

```
SELECT COUNT(DISTINCT customer_id) AS active_customers
FROM orders;
```



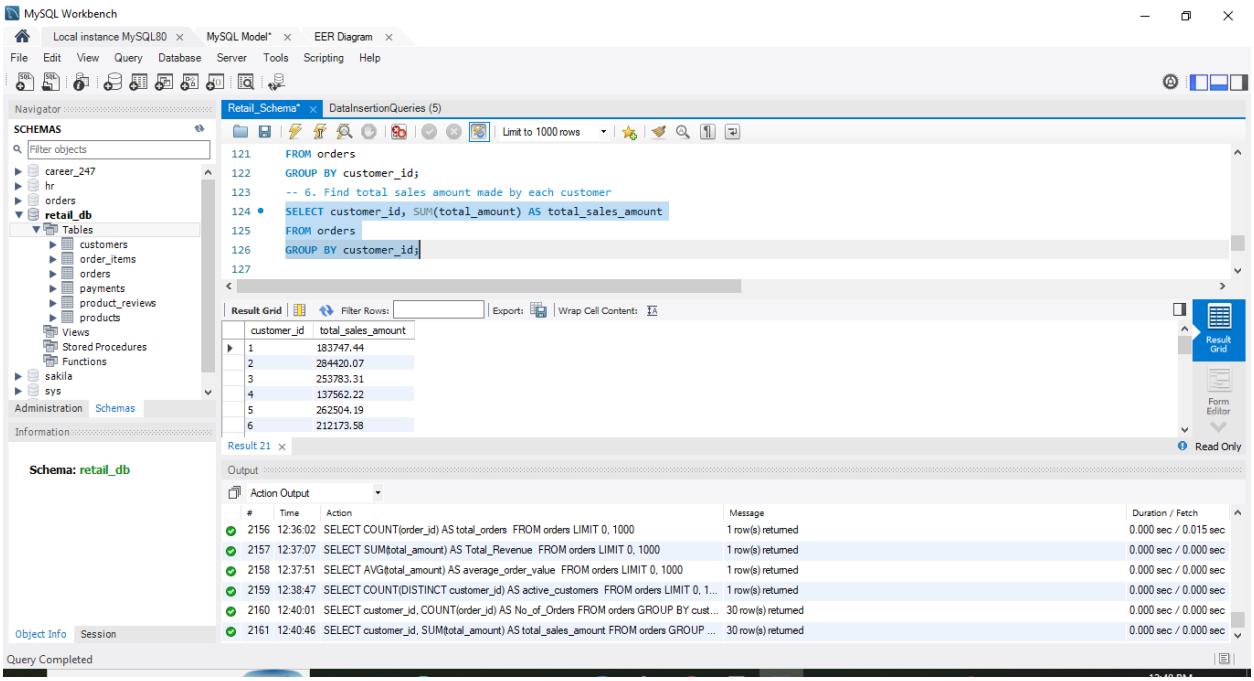
-- 5. Find the number of orders placed by each customer

```
SELECT customer_id, COUNT(order_id) AS No_of_Orders
FROM orders
GROUP BY customer_id;
```



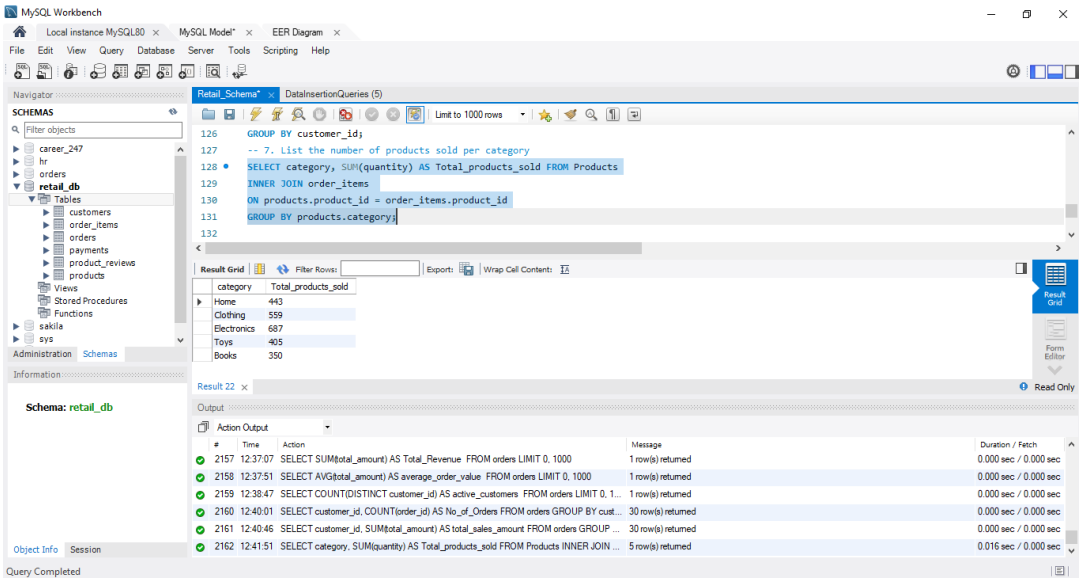
-- 6. Find total sales amount made by each customer

```
SELECT customer_id, SUM(total_amount) AS total_sales_amount
FROM orders
GROUP BY customer_id;
```



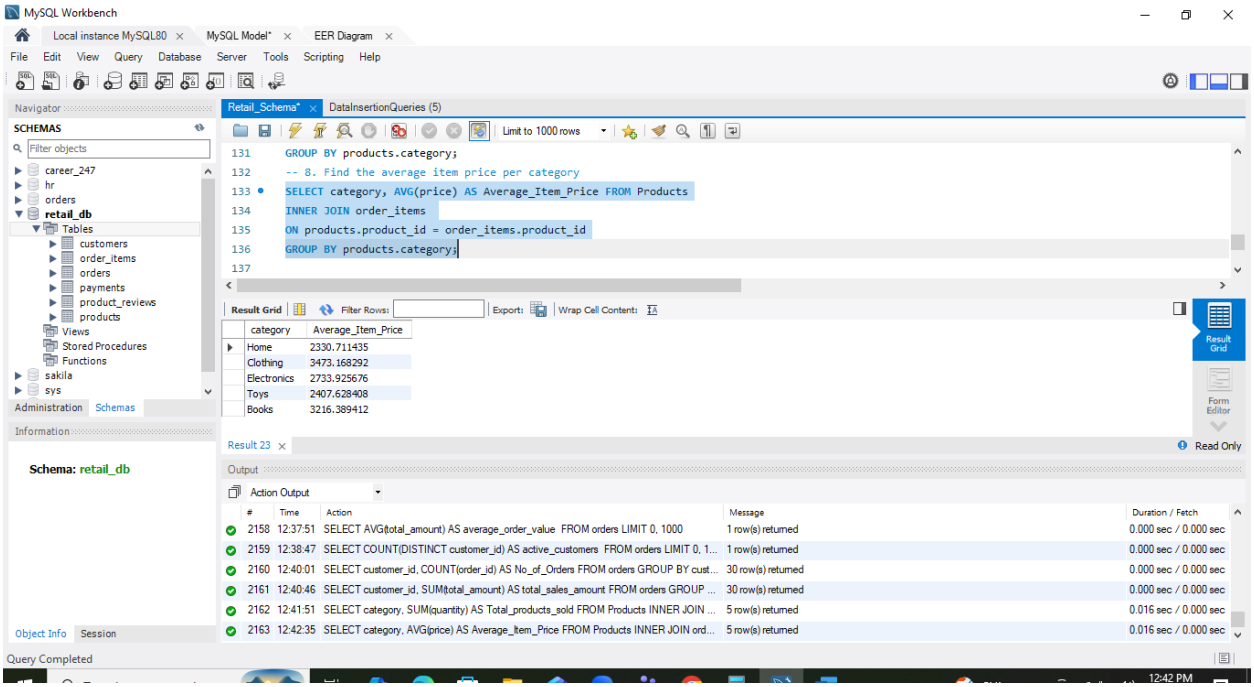
-- 7. List the number of products sold per category

```
SELECT category, SUM(quantity) AS Total_products_sold FROM Products
INNER JOIN order_items
ON products.product_id = order_items.product_id
GROUP BY products.category;
```



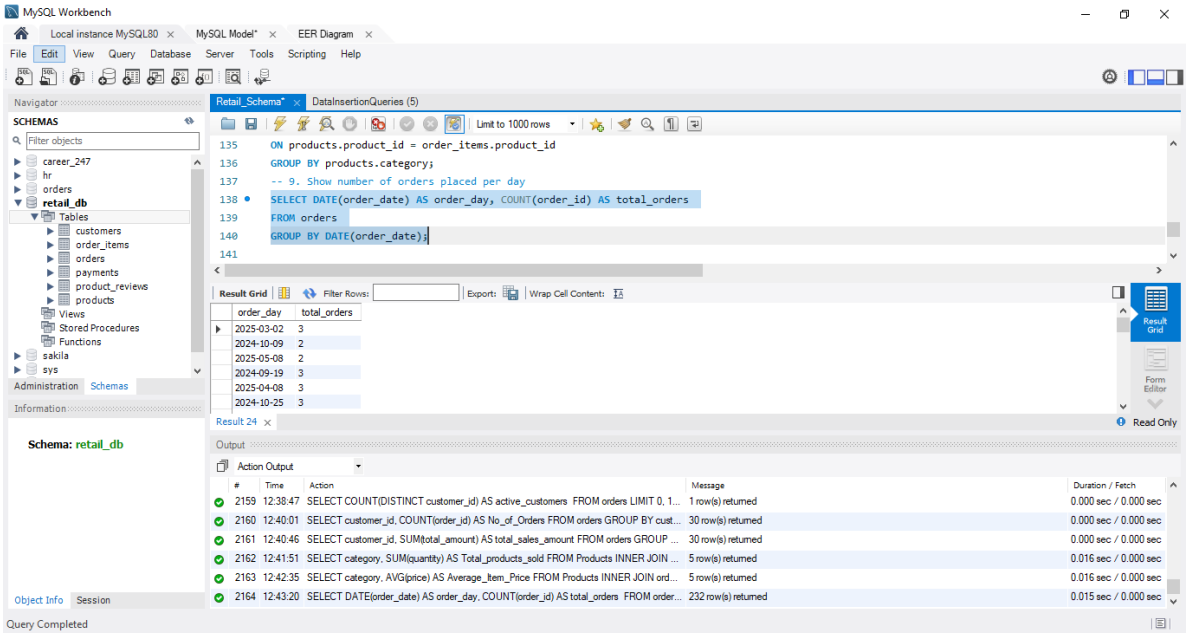
-- 8. Find the average item price per category

```
SELECT category, AVG(price) AS Average_Item_Price FROM Products
INNER JOIN order_items
ON products.product_id = order_items.product_id
GROUP BY products.category;
```



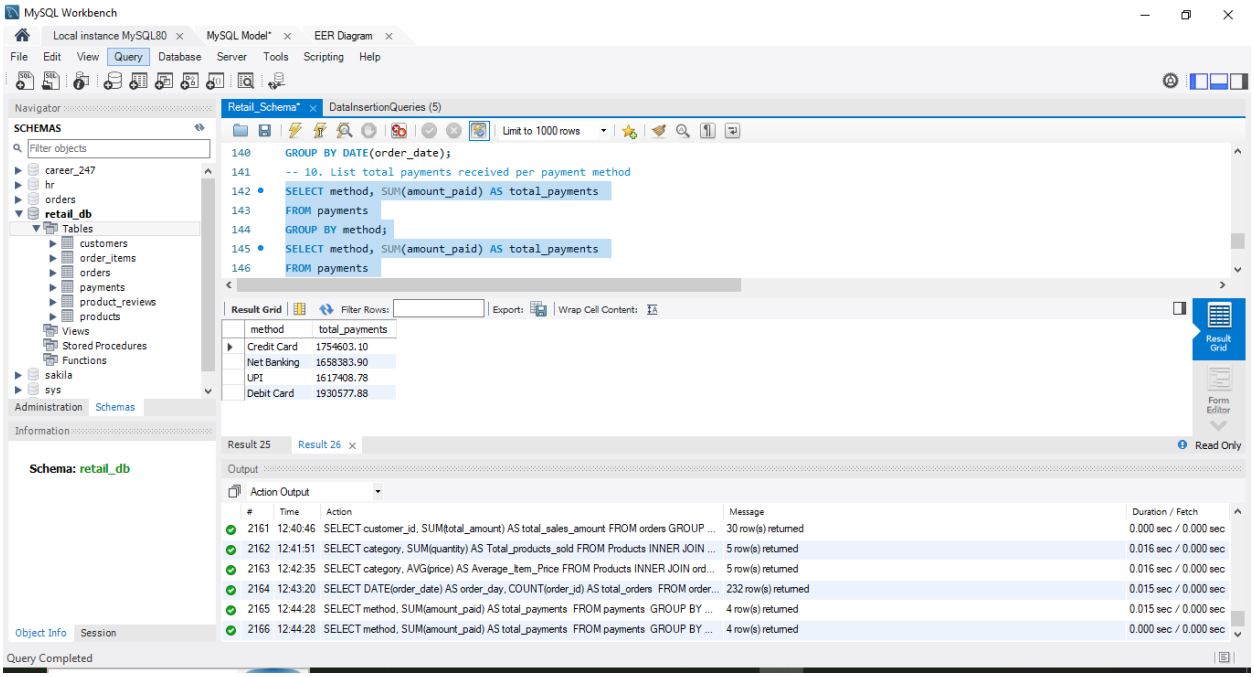
-- 9. Show number of orders placed per day

```
SELECT DATE(order_date) AS order_day, COUNT(order_id) AS total_orders
FROM orders
GROUP BY DATE(order_date);
```



-- 10. List total payments received per payment method

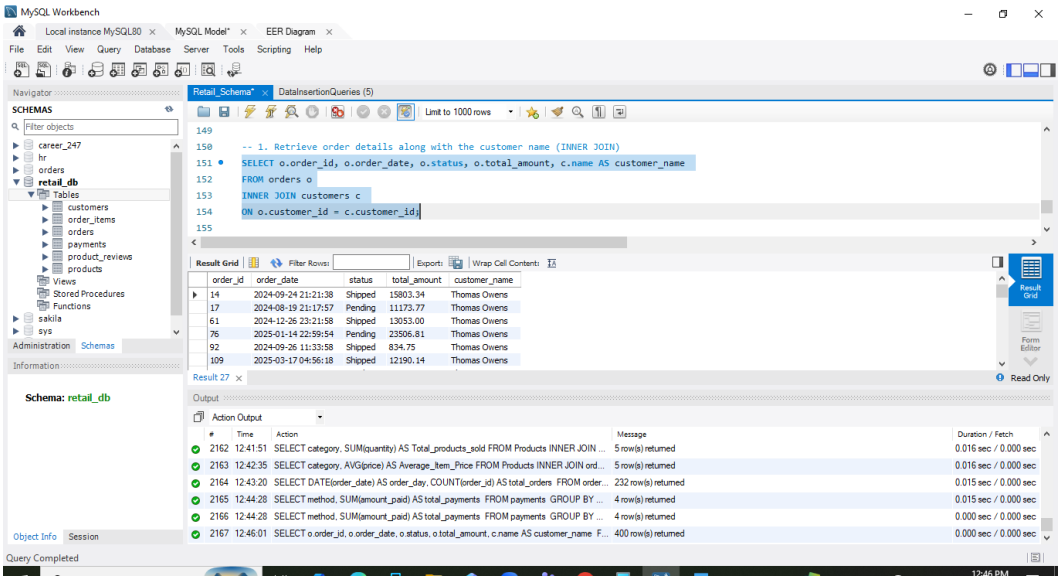
```
SELECT method, SUM(amount_paid) AS total_payments
FROM payments
GROUP BY method;
```



-- Level 4: Multi-Table Queries (JOINS)

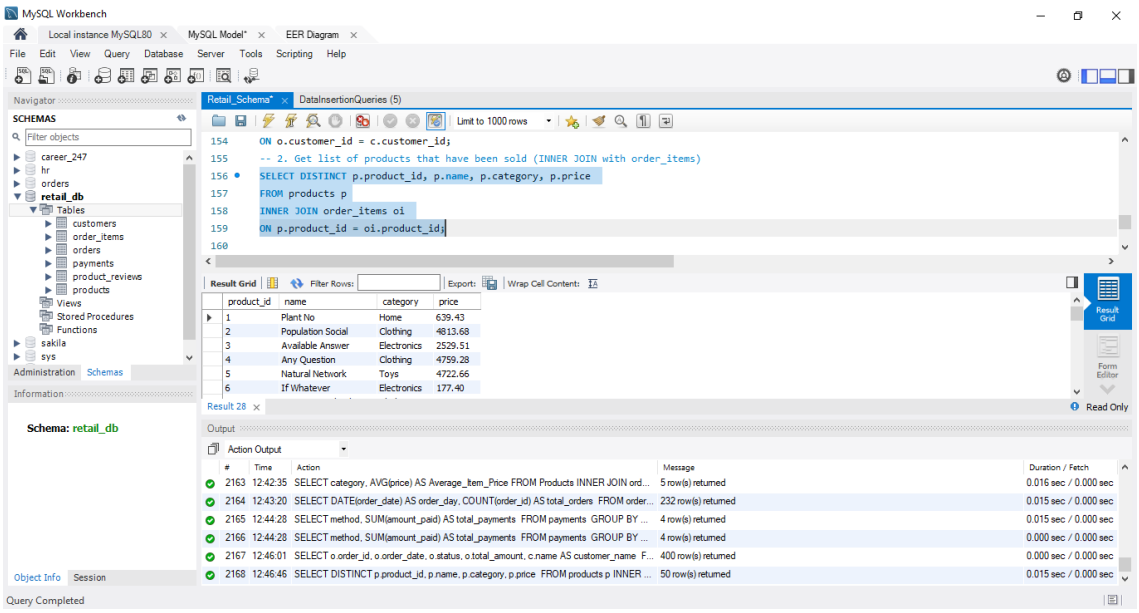
-- 1. Retrieve order details along with the customer name (INNER JOIN)

```
SELECT o.order_id, o.order_date, o.status, o.total_amount, c.name AS customer_name
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;
```



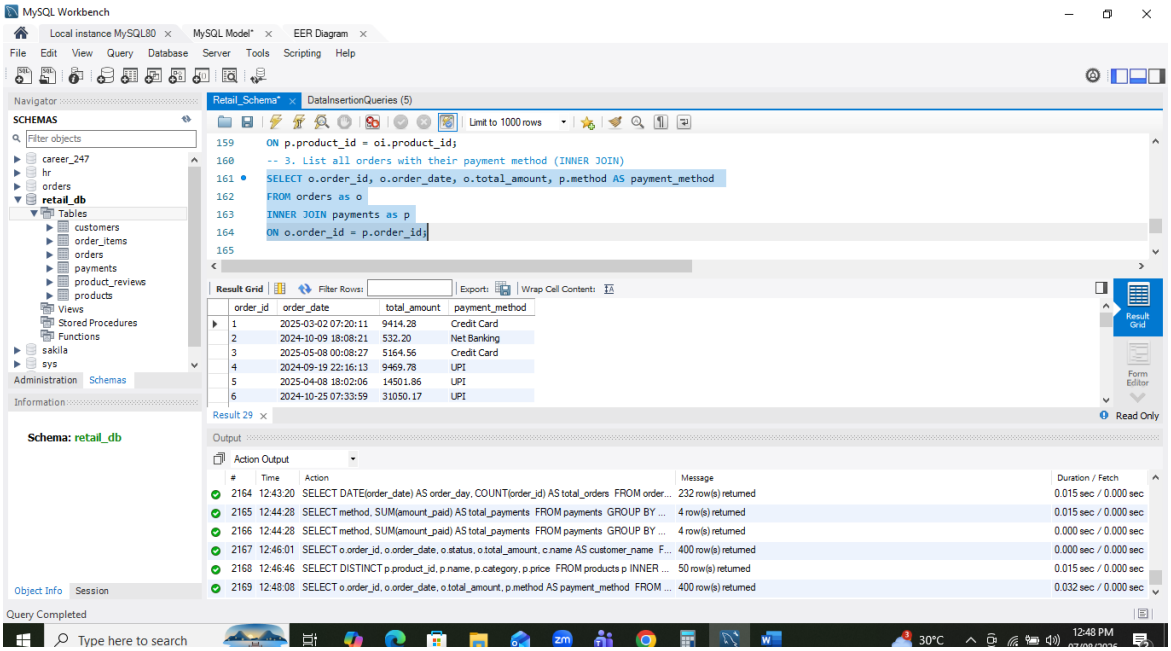
-- 2. Get list of products that have been sold (INNER JOIN with order_items)

```
SELECT DISTINCT p.product_id, p.name, p.category, p.price
FROM products p
INNER JOIN order_items oi
ON p.product_id = oi.product_id;
```



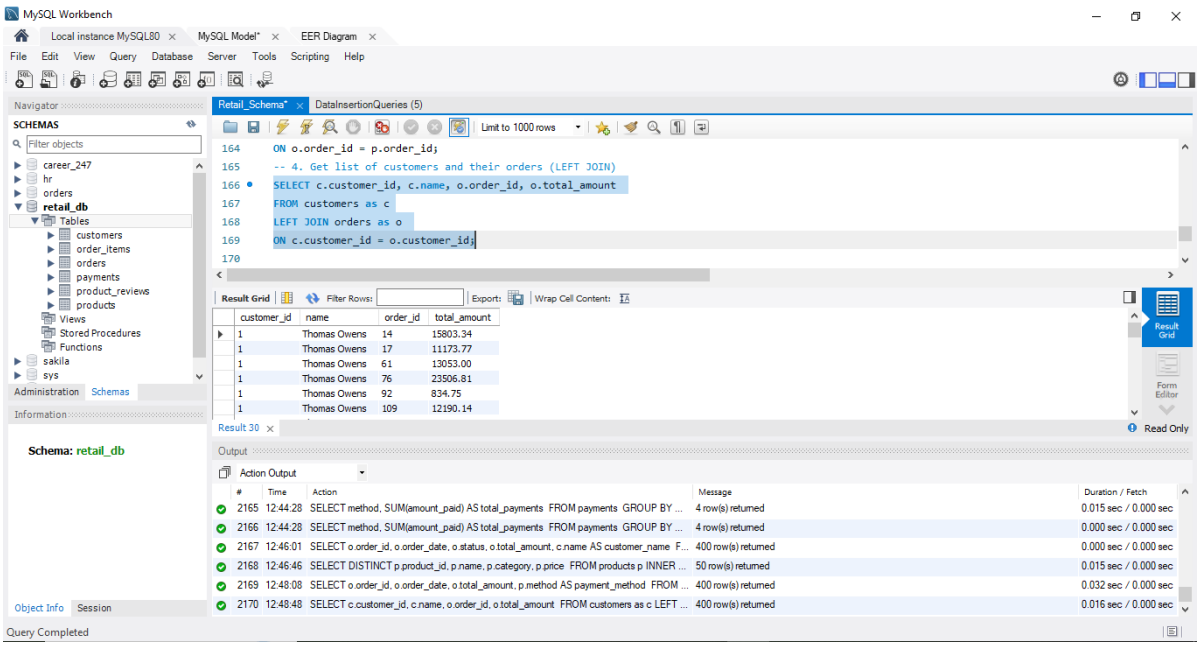
-- 3. List all orders with their payment method (INNER JOIN)

```
SELECT o.order_id, o.order_date, o.total_amount, p.method AS payment_method
FROM orders as o
INNER JOIN payments as p
ON o.order_id = p.order_id;
```



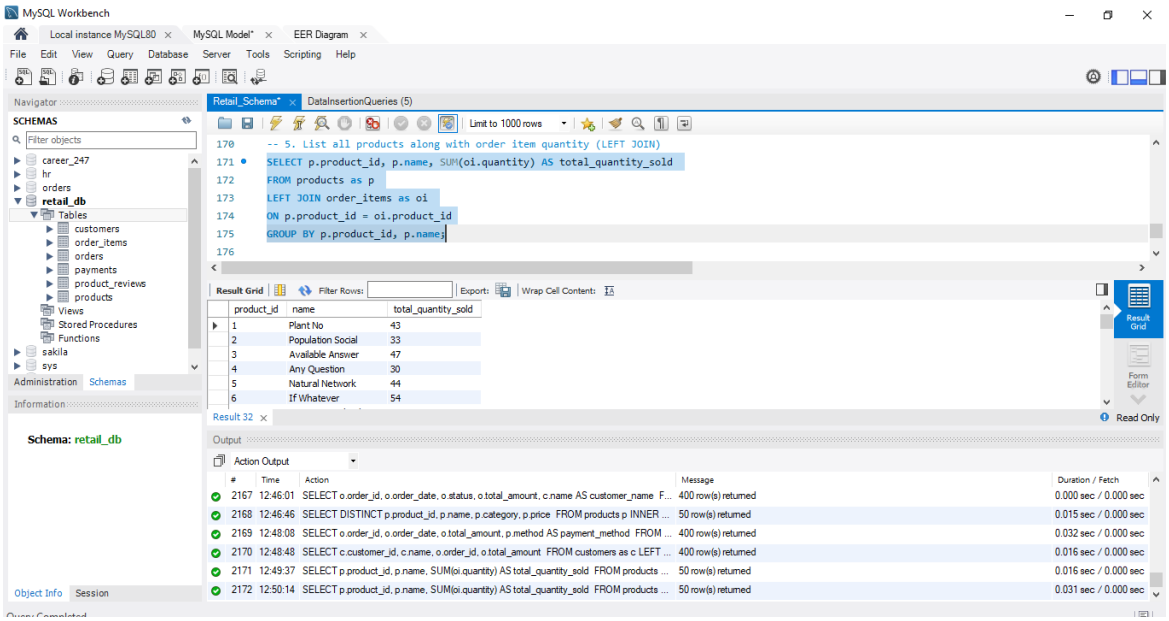
-- 4. Get list of customers and their orders (LEFT JOIN)

```
SELECT c.customer_id, c.name, o.order_id, o.total_amount
FROM customers as c
LEFT JOIN orders as o
ON c.customer_id = o.customer_id;
```



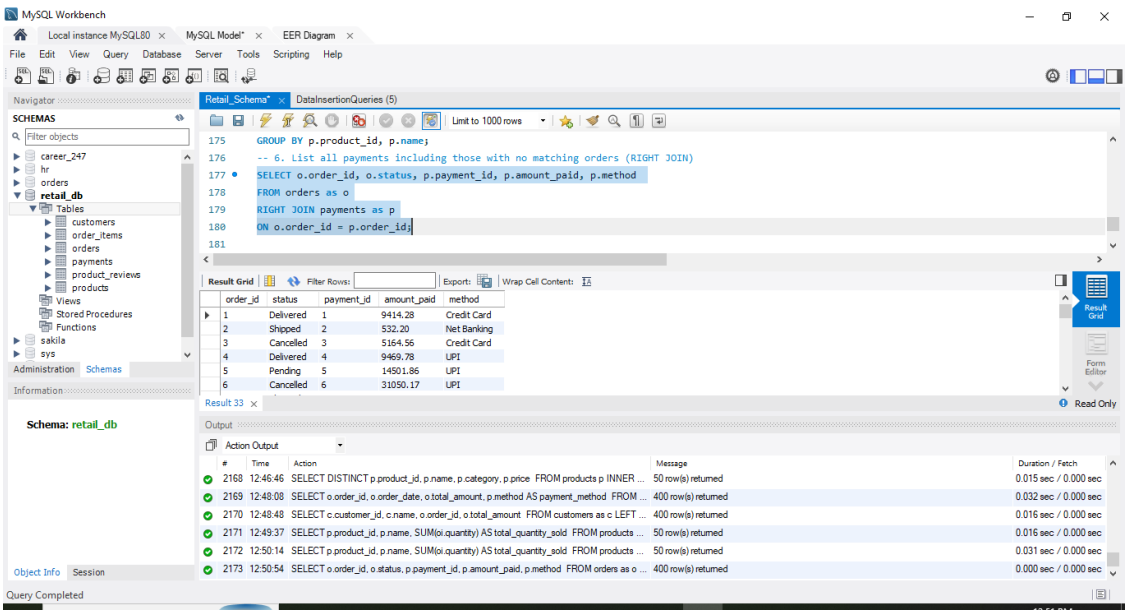
-- 5. List all products along with order item quantity (LEFT JOIN)

```
SELECT p.product_id, p.name, SUM(oi.quantity) AS total_quantity_sold
FROM products as p
LEFT JOIN order_items as oi
ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name;
```



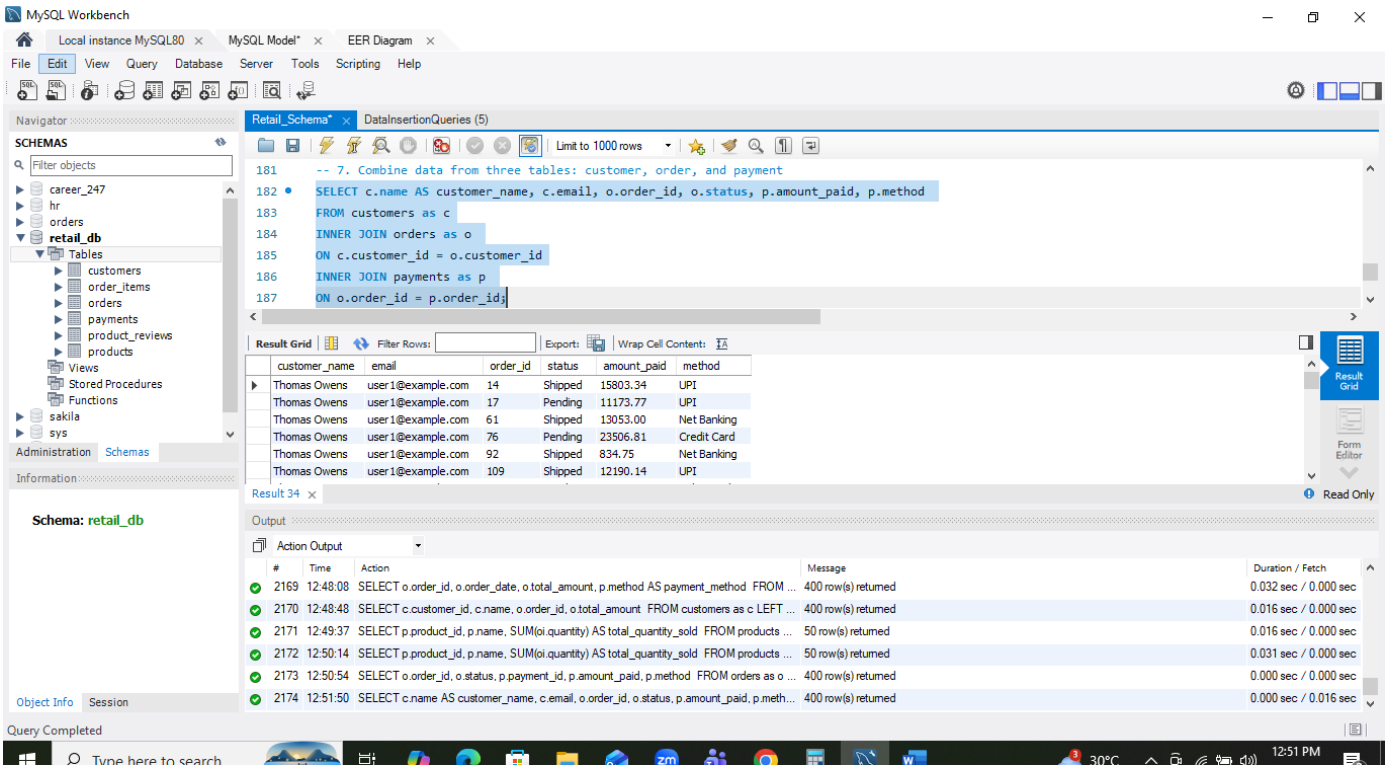
-- 6. List all payments including those with no matching orders (RIGHT JOIN)

```
SELECT o.order_id, o.status, p.payment_id, p.amount_paid, p.method
FROM orders as o
RIGHT JOIN payments as p
ON o.order_id = p.order_id;
```



-- 7. Combine data from three tables: customer, order, and payment

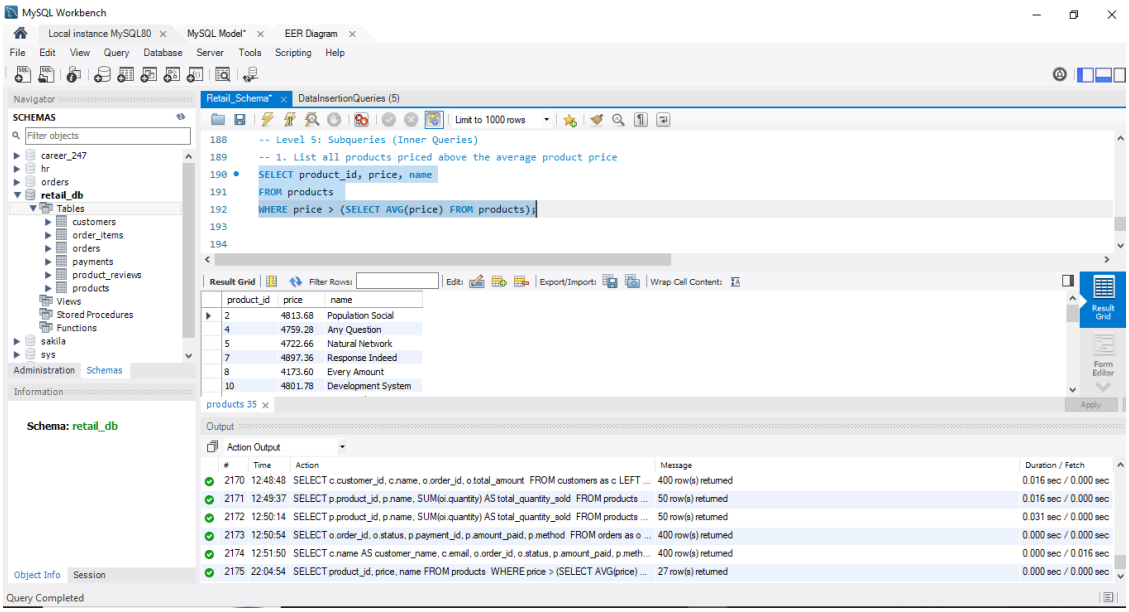
```
SELECT c.name AS customer_name, c.email, o.order_id, o.status, p.amount_paid, p.method
FROM customers as c
INNER JOIN orders as o
ON c.customer_id = o.customer_id
INNER JOIN payments as p
ON o.order_id = p.order_id;
```



-- Level 5: Subqueries (Inner Queries)

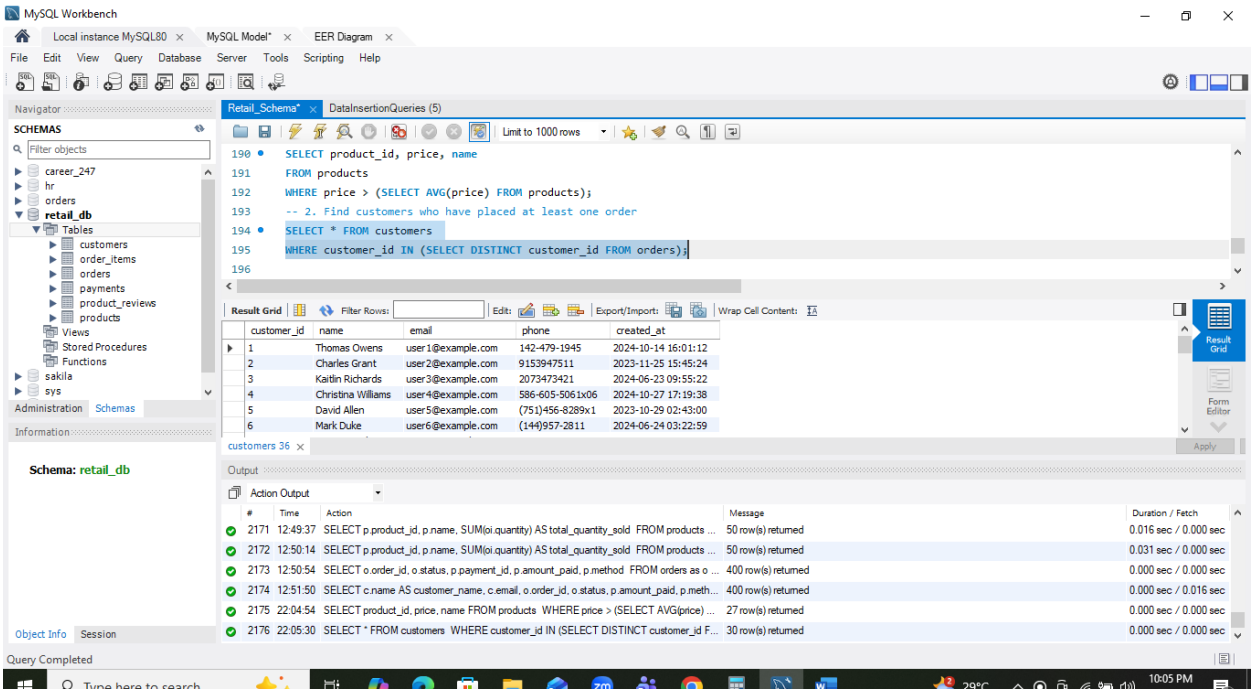
-- 1. List all products priced above the average product price

```
SELECT product_id, price, name
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```



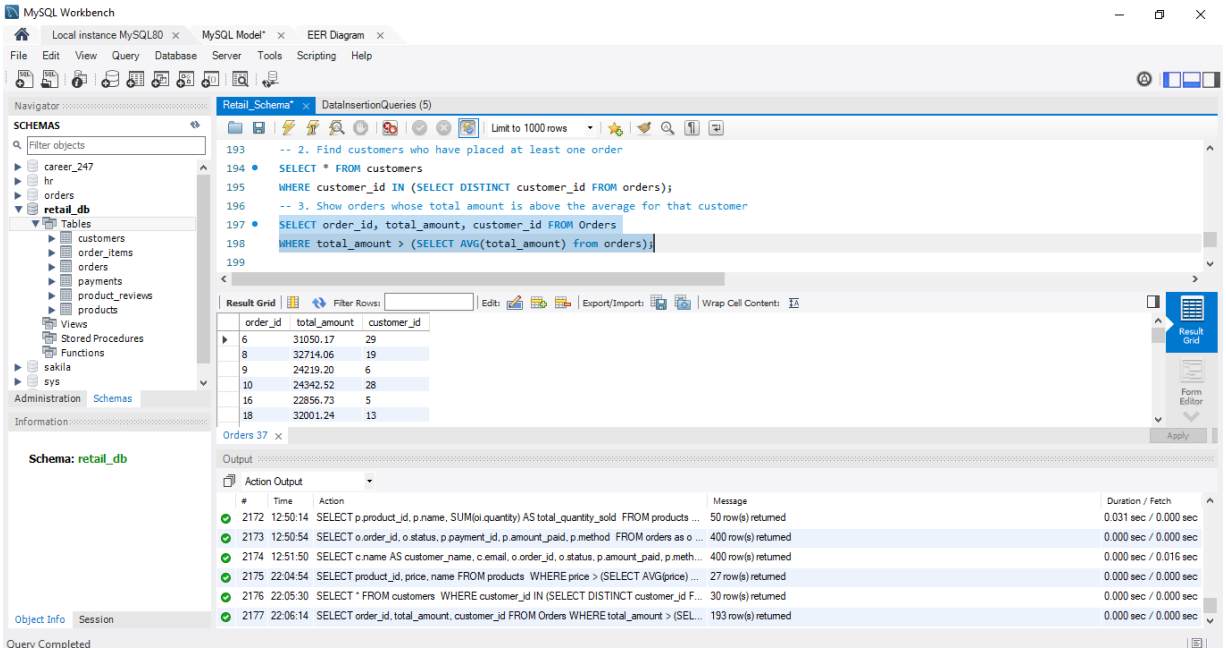
-- 2. Find customers who have placed at least one order

```
SELECT * FROM customers
WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
```



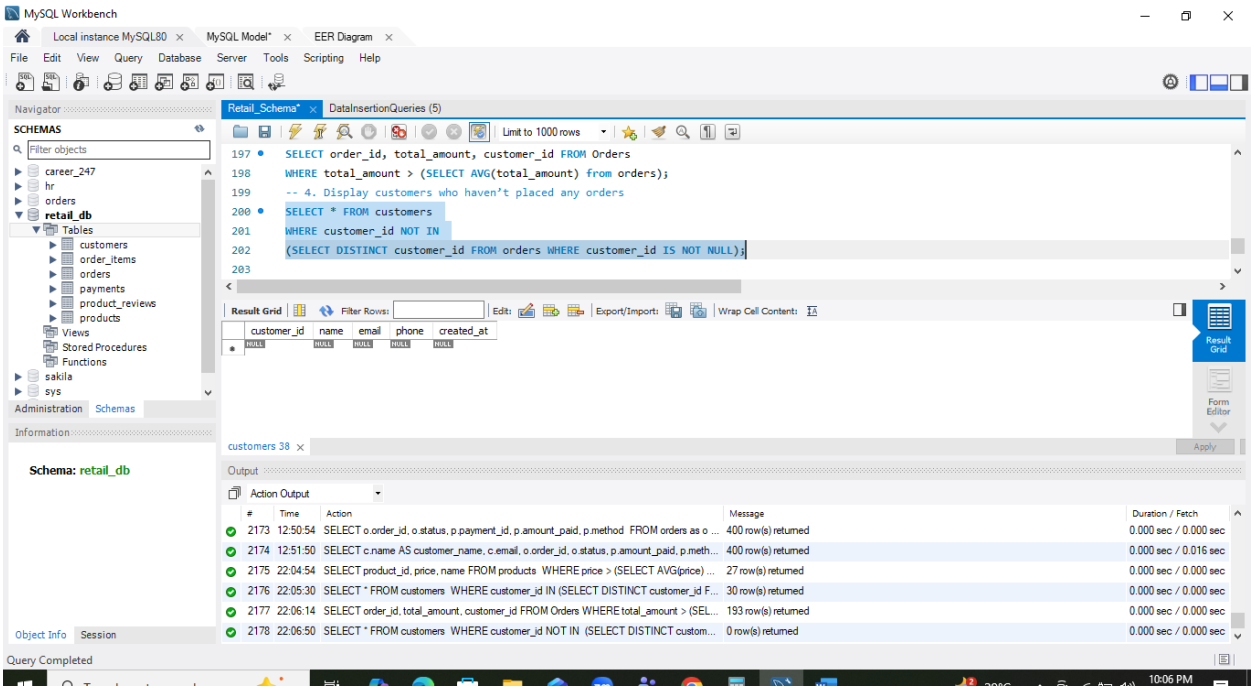
-- 3. Show orders whose total amount is above the average for that customer

```
SELECT order_id, total_amount, customer_id FROM Orders
WHERE total_amount > (SELECT AVG(total_amount) from orders);
```



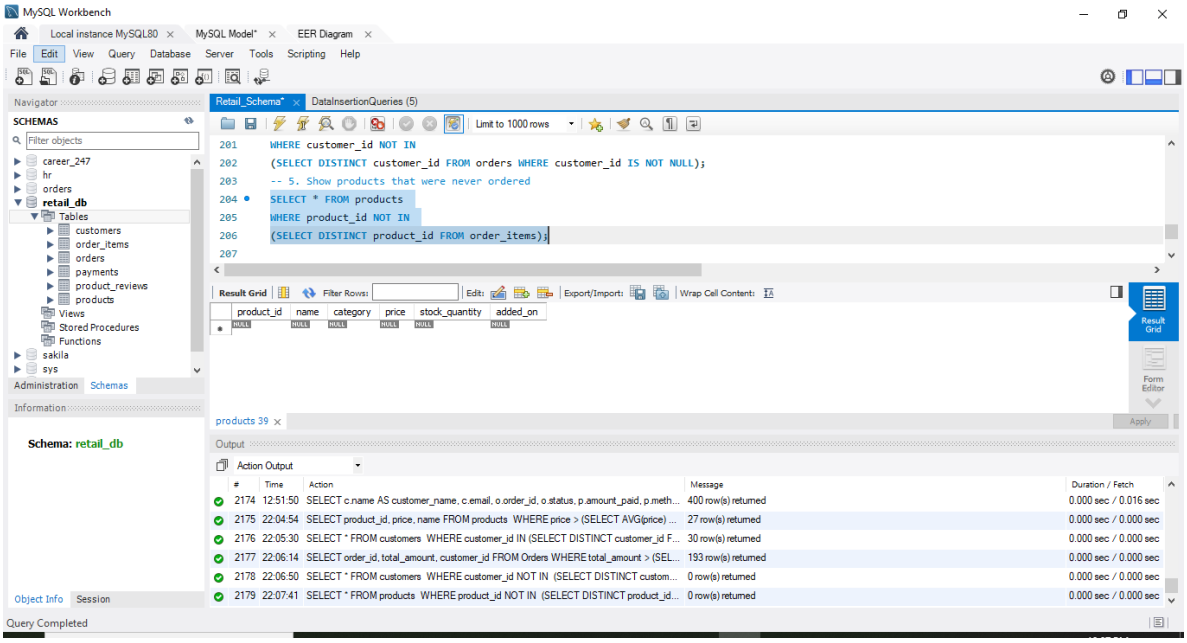
-- 4. Display customers who haven't placed any orders

```
SELECT * FROM customers
WHERE customer_id NOT IN
(SELECT DISTINCT customer_id FROM orders WHERE customer_id IS NOT NULL);
```



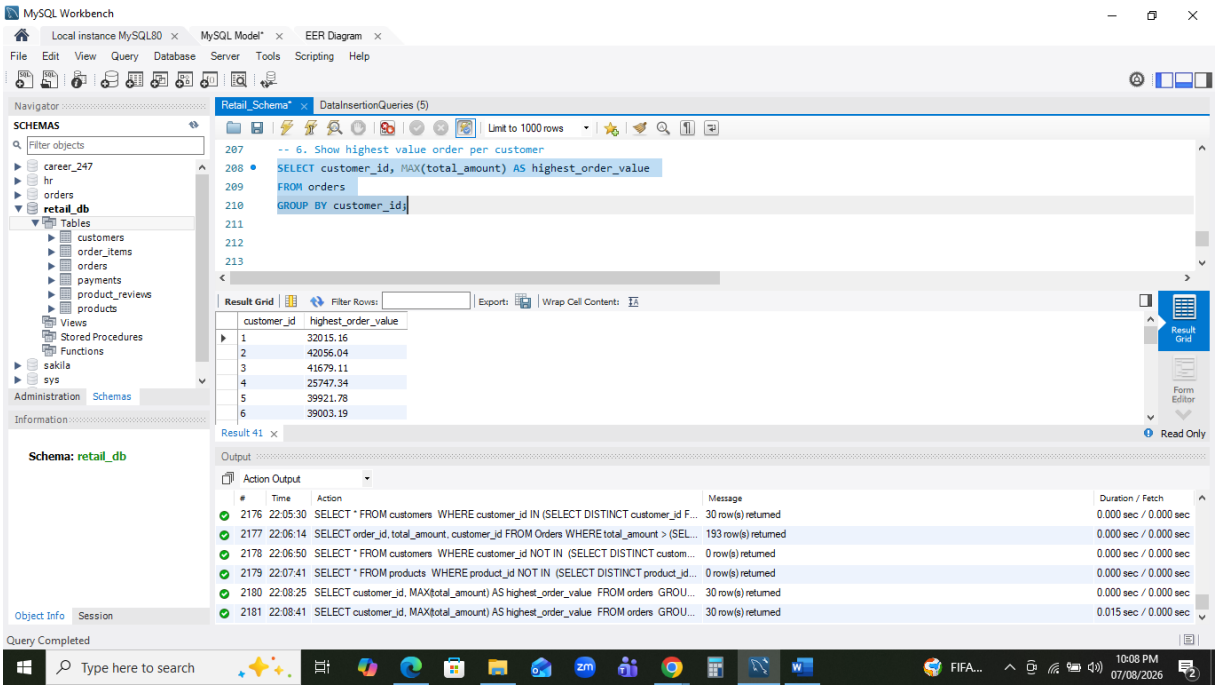
-- 5. Show products that were never ordered

```
SELECT * FROM products
WHERE product_id NOT IN
(SELECT DISTINCT product_id FROM order_items);
```



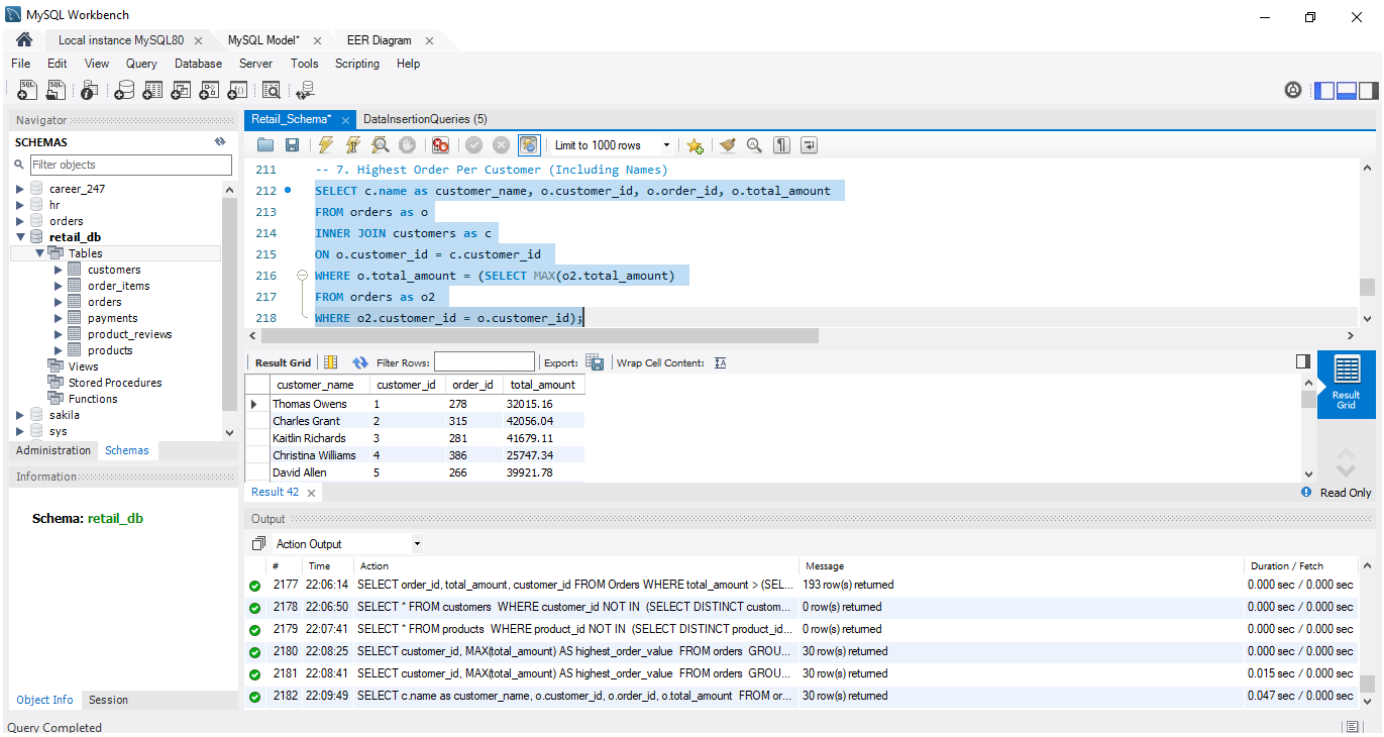
-- 6. Show highest value order per customer

```
SELECT customer_id, MAX(total_amount) AS highest_order_value
FROM orders
GROUP BY customer_id;
```



-- 7. Highest Order Per Customer (Including Names)

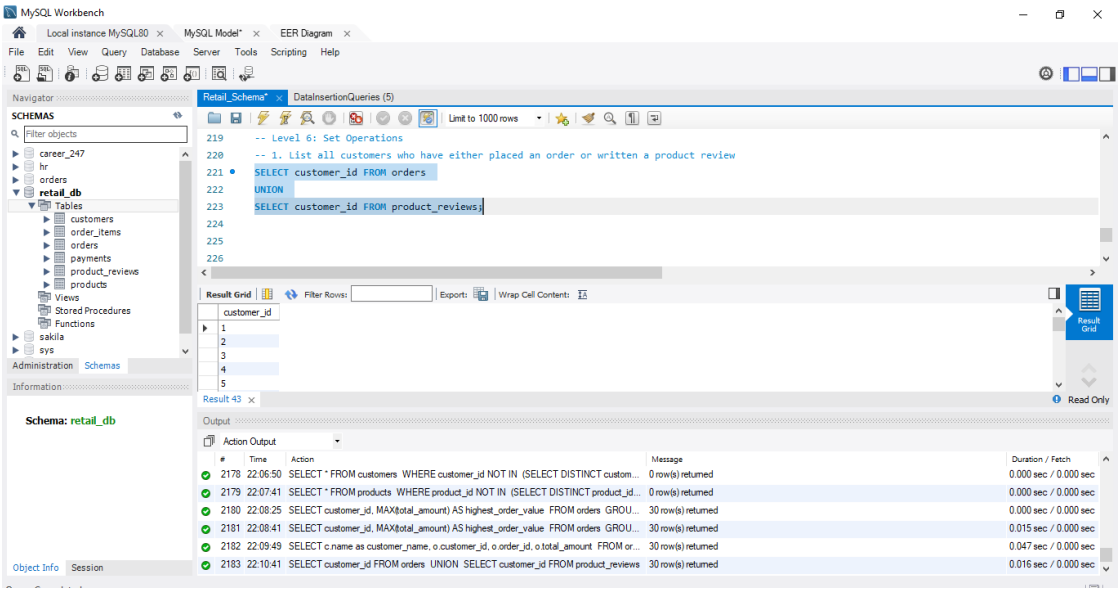
```
SELECT c.name as customer_name, o.customer_id, o.order_id, o.total_amount
FROM orders as o
INNER JOIN customers as c
ON o.customer_id = c.customer_id
WHERE o.total_amount = (SELECT MAX(o2.total_amount)
FROM orders as o2
WHERE o2.customer_id = o.customer_id);
```



-- Level 6: Set Operations

-- 1. List all customers who have either placed an order or written a product review

```
SELECT customer_id FROM orders
UNION
SELECT customer_id FROM product_reviews;
```



-- 2. List all customers who have placed an order as well as reviewed a product

```
SELECT DISTINCT c.customer_id, c.name
FROM customers as c
INNER JOIN orders as o
ON c.customer_id = o.customer_id
INNER JOIN product_reviews as pr
ON c.customer_id = pr.customer_id;
```

